



STARBOOKS

Whiz Challenge

AY 2025-2026

AUTHORS:

Kelly Dumbrigue
Arcielle Marie Gercan
Shandrae Lois Quianzon
Janice Maxene Salipande
NEXUS

CONTRIBUTES TO:

4

QUALITY
EDUCATION



9

INDUSTRY, INNOVATION
AND INFRASTRUCTURE



10

REDUCED
INEQUALITIES



INDIRECTLY AIDS:

8

DECENT WORK AND
ECONOMIC GROWTH



Fully Dressed Use Cases for STARBOOKS Whiz Challenge

Date: January 6, 2026
Draft Version: 1.0

V	Revision Date	Revision Author	Revision Reason	Approved By	Approved Date
1.0	01/05/2026	Janice Maxene Salipande	Original Draft		

Table of Contents

Table of Contents	3
UC-MPA1: Player Register	4
UC-MPA2: Player Login	7
UC-MPA3: Edit Player Profile	9
UC-MPA4: Change Player Password	11
UC-MPA5: Player Logout	15
UC-PG1: Play Whiz Challenge	17
UC-PG2: Play Whiz Memory Match	21
UC-PG3: Play Whiz Puzzle	24
UC-PG4: Play Whiz Battle	27
UC-RR1: Player Requests Reward	33
UC-RR2: Admin Grants Reward	36
UC-MAA1: Admin Login	39
UC-MAA2: Add New Admin	42
UC-MAA3: Edit Admin Profile	45
UC-MAA4: Change Admin Password	48
UC-MAA5: Delete Admin	52
UC-MAA6: Admin Logout	55
UC-MQS1: Add New Question	57
UC-MQS2: Edit Question Info	60
UC-MQS3: Delete Question	63
UC-MQS4: Set Number of Questions	66
UC-MQS5: Set Stars	69
UC-MQS6: Set Timer	72
UC-MPL1: Add New Player	76
UC-MPL2: Edit Player Profile	80
UC-MPL3: Change Player Password	83
UC-MPL4: Delete Player	87
UC-VAD1: View Analytics Dashboard	90
UC-VLB1: Player View Leaderboard	95
UC-VLB2: Admin View Leaderboard	98

UC-MPA1: Player Register

Use Case Name

Player Register

Use Case ID

UC-MPA1

Description

Goal

To allow a new player to create an account in the STARBOOKS system so they can access the game and track their progress.

Actions/Short Summary

- Player navigates to registration page
- Player enters required account information
- System validates the input data
- System creates new player account
- System confirms successful registration and redirects to login page

Trigger

Player clicks "Register here" link from the login page because they do not have an existing account.

Actor(s)

1. **Primary Actor** - Player
2. **Secondary Actors** - System

Assumptions

1. The system is available and accessible via web browser.
2. The player has not previously registered an account.
3. All registration fields have defined validation rules.

Preconditions

1. System is available and displays the login page.
2. Player is not currently logged in.
3. Player does not have an existing account in the system.

Post-conditions

1. A new player account is created and stored in the database.
2. Player receives confirmation of successful registration.

3. System redirects player to the login page.
4. System is ready to accept login credentials for the newly created account.

Scenario 1: New Player Registration

Base Flow

Step	Executor	Action/Description	Data (Input or Output)
1	Player	Clicks "Register here" link from the login page.	Input: Click action
2	System	Displays registration form with required fields.	Output: Registration form with fields (username, password, school name, age, category, sex, region, province, city, avatar)
3	Player	Enters required details into all form fields and clicks "Submit" button.	Input: username, password, school name, age, category, sex, region, province, city, avatar selection
4	System	Validates all input fields: checks that all required fields are filled, username is unique, password meets strength requirements, age is within valid range, all selections are valid options.	Input: All registration data from Step 3
5	System	Creates new player account record in the database.	Output: Unique Player ID, Account creation timestamp
6	System	Displays success message: "Registration successful!"	Output: Success message
7	System	Redirects player to the login page.	Output: Login page display

Alternative Flows

Base Step	Alt Flow Step	Executor	Action/Description	Data (Input and/or Output)
4			Condition: Player enters incomplete registration data	
	1	System	Identifies missing required fields.	Input: Incomplete form data
	2	System	Displays error message: "Please fill in all required fields"	Output: Error message

	3	System	Returns to Step 3 of base flow.	Output: Registration form with existing data preserved
4			Condition: Username already exists	
	1	System	Checks username against existing accounts in database and finds a match.	Input: Username from registration form
	2	System	Displays error message: "Username already exists."	Output: Error message
	3	System	Returns to Step 3 of base flow.	Output: Registration form with data preserved
4			Condition: Password does not meet security requirements	
	1	System	Validates password against security rules (minimum length, complexity requirements).	Input: Password from registration form
	2	System	Displays error message: "Password must be at least 8 characters and include uppercase, lowercase, and numbers."	Output: Error message, password requirements
	3	System	Returns to Step 3 of base flow.	Output: Registration form with data preserved
4			Condition: Invalid input format	
	1	System	Validates input formats (age is numeric and within range, selections are valid options, etc.).	Input: Registration form data
	2	System	Displays error message specifying which field(s) have invalid format.	Output: Error message
	3	System	Returns to Step 3 of base flow.	Output: Registration form with existing data preserved

UC-MPA2: Player Login

Use Case Name

Player Login

Use Case ID

UC-MPA2

Description

Goal

To allow a registered player to authenticate and access the STARBOOKS system using their credentials, enabling them to play the game and access their account features.

Actions/Short Summary

- System displays login form
- Player enters username and password
- System verifies credentials against database
- System authenticates player and establishes session
- System redirects player to homepage

Trigger

Player opens the STARBOOKS application or navigates to the login page because they want to access their account and play the game.

Actor(s)

1. **Primary Actor** - Player
2. **Secondary Actors** - System

Assumptions

1. The system is available and accessible.
2. The player has already registered an account.
3. The system has a secure authentication mechanism in place.
4. Player credentials are stored securely in the database.

Preconditions

1. System is available and operational.
2. Player has a registered account in the system.
3. Player is not currently logged in.
4. Login page is accessible.

Post-conditions

1. Player is authenticated and logged into the system.
2. Player session is created and active.
3. Player is redirected to the homepage.
4. System is ready to process player actions and requests.

Scenario 1: Successful Login with Valid Credentials

Base Flow

Step	Executor	Action/Description	Data (Input or Output)
1	System	Displays login form with username and password fields.	Output: Login form (username field, password field, "Login" button)
2	Player	Enters username and password into the respective fields and clicks "Login" button.	Input: username, password
3	System	Retrieves the entered credentials and validates format (fields not empty, proper data types).	Input: username, password from Step 2
4	System	Queries database to verify credentials match an existing account.	Input: username, password; Output: Account record (Player ID, username, hashed password, profile data)
5	System	Compares entered password with stored hashed password.	Input: Entered password, stored hashed password
6	System	Creates player session with session token.	Output: Session ID, session token, Player ID, login timestamp
7	System	Redirects player to the homepage.	Output: Homepage display with player profile and game options

Alternative Flows

Base Step	Alt Flow Step	Executor	Action/Description	Data (Input and/or Output)
4			Condition: Invalid credentials (username not found or password incorrect)	
	1	System	Queries database and finds no matching username OR password does not match.	Input: username, password; Output: No match found or password mismatch

	2	System	Displays error message: "Invalid username or password."	Output: Error message
	3	System	Returns to Step 2 of base flow.	Output: Login form with empty fields
3			Condition: Empty login fields	
	1	System	Validates that username and password fields are not empty.	Input: username field, password field
	2	System	Displays error message: "Please enter both username and password."	Output: Error message
	3	System	Returns to Step 2 of base flow.	Output: Login form

UC-MPA3: Edit Player Profile

Use Case Name

Edit Player Profile

Use Case ID

UC-MPA3

Description

Goal

To allow a logged-in player to update their account information in the STARBOOKS system, ensuring their profile reflects current and accurate personal details.

Actions/Short Summary

- Player navigates to Edit Profile section
- System displays current profile information in editable form
- Player modifies desired fields
- System validates the updated information
- System saves changes to database
- System confirms successful profile update

Trigger

Player clicks on their avatar/profile icon and selects "Edit Profile" because they want to update their account information.

Actor(s)

1. **Primary Actor** - Player
2. **Secondary Actors** - System

Assumptions

1. The system is available and player session is active.
2. All profile fields have defined validation rules.

Preconditions

1. Player is logged into the system (has active session).
2. Player has a valid account in the database.
3. System can retrieve current player profile data.
4. Edit Profile page is accessible to the player.

Post-conditions

1. Player profile information is updated in the database.
2. System displays confirmation message to the player.
3. Updated profile information is reflected across the system.
4. Player remains logged in with active session.
5. System is ready to accept additional player actions.

Scenario 1: Player Updates Profile Information Successfully**Base Flow**

Step	Executor	Action/Description	Data (Input or Output)
1	Player	Clicks on "Edit Profile" beside the profile section.	Input: Click action on Edit Profile option
2	System	Retrieves current player profile data from database.	Input: Player ID from session; Output: Current profile data (username, school name, age, category, sex, region, province, city, avatar)
3	System	Displays Edit Profile form with fields pre-populated with current information.	Output: Edit Profile form with current data populated in editable fields
4	Player	Modifies one or more editable fields (school name, age, category, sex, region, province, city, avatar) and clicks "Save Changes" button.	Input: Updated values for modified fields
5	System	Validates all input fields: checks that required fields are not empty, age is within valid range, selections are valid options, data formats are correct.	Input: Updated profile data from Step 4

6	System	Compares new data with existing data and identifies changed fields.	Input: New profile data, existing profile data; Output: List of changed fields
7	System	Updates player profile record in database with new information.	Input: Player ID, updated field values; Output: Updated profile record with modification timestamp
8	System	Displays success message: "Profile Updated!"	Output: Success message
9	System	Refreshes the Edit Profile page showing the updated information.	Output: Edit Profile form with newly updated data

Alternative Flows

Base Step	Alt Flow Step	Executor	Action/Description	Data (Input and/or Output)
5			Condition: Invalid or empty required fields	
	1	System	Identifies required fields that are empty or have invalid format.	Input: Profile form data from Step 4
	2	System	Displays validation error message: "Please complete all required fields" or "[] is required"	Output: Error message
	3	System	Returns to Step 4 of base flow.	Output: Edit Profile form with entered data preserved

UC-MPA4: Change Player Password**Use Case Name**

Change Player Password

Use Case ID

UC-MPA4

Description**Goal**

To allow a logged-in player to change their account password in the STARBOOKS system, ensuring account security and allowing players to update their credentials when needed.

Actions/Short Summary

- Player navigates to Change Password section
- System displays password change form
- Player enters current password and new password
- System verifies current password is correct
- System validates new password meets security requirements
- System updates password in database
- System confirms successful password change

Trigger

Player clicks "Change Password" button from the Edit Profile page because they want to update their account password for security reasons or personal preference.

Actor(s)

1. **Primary Actor** - Player
2. **Secondary Actors** - System

Assumptions

1. The system is available and player session is active.
2. Passwords are stored securely using encryption/hashing in the database.
3. The system has defined password security requirements.
4. Player remembers their current password.

Preconditions

1. Player is logged into the system (has active session).
2. Player has accessed the Edit Profile page.
3. Player has a valid account with an existing password.
4. Change Password form/dialog is accessible from Edit Profile page.

Post-conditions

1. Player's password is updated in the database with new hashed password.
2. Old password is replaced and can no longer be used for login.
3. System displays confirmation message to the player.
4. Player remains logged in with current session.
5. System is ready to accept additional player actions.

Scenario 1: Player Successfully Changes Password

Base Flow

Step	Executor	Action/Description	Data (Input or Output)
1	Player	Clicks "Change Password" button from the Edit Profile page.	Input: Click action on Change Password option
2	System	Displays Change Password form/dialog with three fields: Current Password, New Password, Confirm New Password.	Output: Password change form (current password field, new password field, confirm new password field, "Update Password" button, "Cancel" button)
3	Player	Enters current password, new password, and confirms new password by re-entering it. Clicks "Update Password" button.	Input: current password, new password, confirm new password
4	System	Validates that all three fields are not empty.	Input: Form data from Step 3
5	System	Retrieves player's stored hashed password from database.	Input: Player ID from session; Output: Stored hashed password
6	System	Verifies entered current password matches the stored hashed password.	Input: Entered current password, stored hashed password
7	System	Validates new password matches confirm new password field (ensures no typos).	Input: New password, confirm new password
8	System	Validates new password meets security requirements (minimum 8 characters, includes uppercase, lowercase, and numbers).	Input: New password; Output: Validation result
9	System	Hashes the new password using secure hashing algorithm.	Input: New password (plain text); Output: Hashed new password
10	System	Updates player's password in database with the new hashed password.	Input: Player ID, new hashed password; Output: Updated password record
11	System	Displays success message: "Password is updated!"	Output: Success message
12	System	Closes the Change Password form and returns player to Edit Profile page.	Output: Edit Profile page display

Alternative Flows

Base Step	Alt Flow Step	Executor	Action/Description	Data (Input and/or Output)
4			Condition: Empty password fields	

	1	System	Identifies one or more password fields are empty.	Input: Form data from Step 3
	2	System	Displays error message: "Please fill in all password fields."	Output: Error message
	3	System	Returns to Step 3 of base flow.	Output: Password change form
6			Condition: Incorrect current password	
	1	System	Compares entered current password with stored hashed password and finds no match.	Input: Entered current password, stored hashed password
	2	System	Displays error message: "Current password is incorrect."	Output: Error message
	3	System	Clears all password fields for security.	Output: Password change form with empty fields
	4	System	Returns to Step 3 of base flow.	Output: Password change form
7			Condition: New password and confirm password do not match	
	1	System	Compares new password field with confirm new password field and finds mismatch.	Input: New password, confirm new password
	2	System	Displays error message: "New passwords do not match."	Output: Error message
	3	System	Clears new password and confirm new password fields.	Output: Password change form with current password preserved
	4	System	Returns to Step 3 of base flow.	Output: Password change form
8			Condition: New password does not meet security requirements	
	1	System	Validates new password against security rules and identifies violations.	Input: New password
	2	System	Displays error message: "Password must be at least 8 characters and include uppercase, lowercase, and numbers."	Output: Error message with specific requirements

	3	System	Clears new password and confirm new password fields.	Output: Password change form with current password preserved
	4	System	Returns to Step 3 of base flow.	Output: Password change form
8			Condition: New password is same as current password	
	1	System	Compares new password with current password and finds they are identical.	Input: New password, current password
	2	System	Displays error message: "New password must be different from current password."	Output: Error message
	3	System	Clears new password and confirm new password fields.	Output: Password change form with current password preserved
	4	System	Returns to Step 3 of base flow.	Output: Password change form

UC-MPA5: Player Logout

Use Case Name

Player Logout

Use Case ID

UC-MPA5

Description

Goal

To allow a logged-in player to safely end their session and exit the STARBOOKS system, ensuring their account security by terminating access.

Actions/Short Summary

- Player initiates logout action
- System displays confirmation dialog
- Player confirms logout intention
- System terminates player session
- System clears session data
- System redirects player to login page

Trigger

Player clicks on their avatar/profile icon and selects "Logout" from the dropdown menu because they want to end their session and secure their account.

Actor(s)

1. **Primary Actor** - Player
2. **Secondary Actors** - System

Assumptions

1. The system is available and player session is active.
2. The system maintains session data that can be terminated.
3. Logout is accessible from all pages within the system via avatar menu.
4. Unsaved changes in other areas are handled separately.

Preconditions

1. Player is logged into the system (has active session).
2. Player session is valid and not expired.
3. Logout option is accessible from the avatar/profile dropdown menu.

Post-conditions

1. Player session is terminated and removed from active sessions.
2. Session data is cleared from server and client.
3. Player is logged out of the system.
4. Player is redirected to the login page.
5. System no longer recognizes player as authenticated.
6. Player must login again to access the system.

Scenario 1: Player Successfully Logs Out**Base Flow**

Step	Executor	Action/Description	Data (Input or Output)
1	Player	Clicks on avatar/profile icon.	Input: Click action on avatar
2	System	Displays confirmation dialog: "Are you sure you want to logout?" with "Yes" and "No" buttons.	Output: Confirmation dialog (message, Yes button, No button)
3	Player	Clicks "Yes" button to confirm logout.	Input: Click action on Yes button
4	System	Displays logout success message: "You have been logged out successfully."	Output: Success message

5	System	Redirects player to the login page.	Output: Login page display
---	--------	-------------------------------------	----------------------------

Alternative Flows

Base Step	Alt Flow Step	Executor	Action/Description	Data (Input and/or Output)
2			Condition: Player cancels logout	
	1	Player	Clicks "No" button or closes the confirmation dialog.	Input: Click action on No button or dialog close
	2	System	Closes the confirmation dialog without taking any action.	Input: None
	3	System	Returns player to the previous page they were viewing.	Output: Previous page display with player still logged in
	4	Player session remains active and valid.	Output: Active session maintained	
	5	Use case ends.		

UC-PG1: Play Whiz Challenge**Use Case Name**

Play Whiz Challenge

Use Case ID

UC-PG1

Description**Goal**

To allow a logged-in player to play the Whiz Challenge game mode by answering timed questions in a selected category and difficulty level, potentially earning badges based on performance.

Actions/Short Summary

- Player selects Whiz Challenge game mode from homepage
- Player chooses category (Science or Math)
- Player chooses difficulty level (Easy, Average, Difficult)
- System loads appropriate questions

- Player answers questions within time limit
- System evaluates answers and calculates score
- System updates player stats and awards badges if criteria met

Trigger

Player clicks on "Whiz Challenge" game mode from the homepage because they want to test their knowledge and earn badges through a timed quiz challenge.

Actor(s)

1. **Primary Actor** - Player
2. **Secondary Actors** - System

Assumptions

1. The system is available and player session is active.
2. Question bank contains sufficient questions for all categories and difficulty levels.
3. Timer mechanism functions correctly and consistently.
4. Badge criteria are clearly defined in the system (100% correct answers for badge award).

Preconditions

1. Player is logged into the system (has active session).
2. Player has a valid account in the database.
3. Question database contains questions for selected category and difficulty.
4. System can track and update player statistics.
5. Player is on the homepage or game selection screen.

Post-conditions

1. Player's game performance is recorded.
2. Player statistics are updated in the database.
3. Badge is awarded and added to player profile if all questions answered correctly.
4. Game results are displayed to player.
5. System is ready to accept next player action.

Scenario 1: Player Completes Game and Earns Badge**Base Flow**

Step	Executor	Action/Description	Data (Input or Output)
1	Player	Clicks "Whiz Challenge" button/card on the homepage.	Input: Click action on Whiz Challenge

2	System	Displays category selection screen with available categories.	Output: Category selection screen (Science button, Math button)
3	Player	Selects a category (Science or Math) by clicking on it.	Input: Category selection (Science OR Math)
4	System	Records selected category and displays difficulty level selection screen.	Input: Category from Step 3; Output: Difficulty selection screen (Easy, Average, Difficult buttons)
5	Player	Selects a difficulty level (Easy, Average, or Difficult) by clicking on it.	Input: Difficulty level selection (Easy OR Average OR Difficult)
6	System	Retrieves question set from database based on selected category and difficulty.	Input: Category, Difficulty level; Output: Question set (question IDs, question text, answer options, correct answers)
7	System	Initializes game session with timer settings based on difficulty level and displays first question.	Output: Game screen (question counter, timer display, question text, answer options)
8	System	Starts countdown timer.	Output: Timer starts counting down
9	Player	Reads question and selects an answer option.	Input: Selected answer for current question
10	System	Records the answer with timestamp and loads next question.	Input: Answer selection, time remaining; Output: Next question display
11	System	Repeats Steps 9-10 until all questions are answered.	Input/Output: Answer submissions and question displays
12	System	Calculates total score by comparing player answers with correct answers.	Input: Player answers, correct answers; Output: Score
13	System	Evaluates badge criteria: checks if player answered all questions correctly.	Input: Score, accuracy percentage; Output: Badge eligibility status (TRUE)
14	System	Awards badge for the completed difficulty level.	Input: Category, Difficulty level, Player ID; Output: Badge record
15	System	Updates player statistics in database.	Input: Player ID, game results; Output: Updated player stats record
16	System	Displays results screen.	Output: Results screen

Alternative Flows

Base Step	Alt Flow Step	Executor	Action/Description	Data (Input and/or Output)
-----------	---------------	----------	--------------------	----------------------------

8-11			Condition: Timer expires before all questions answered	
	1	System	Detects timer has reached zero.	Input: Timer value = 0
	2	System	Automatically submits all answered questions and marks unanswered questions as incorrect.	Input: Answered questions, unanswered questions; Output: Complete answer set with unanswered marked as wrong
	3	System	Proceeds to Step 12 of base flow to calculate score.	
13			Condition: Player does not answer all questions correctly	
	1	System	Evaluates badge criteria and determines player did not achieve 100% accuracy.	Input: Score, accuracy percentage; Output: Badge eligibility status (FALSE)
	2	System	Does NOT award a badge.	Input: None
	3	System	Updates player statistics without adding a badge.	Input: Player ID, game results; Output: Updated player stats record (no badge added)
	4	System	Displays results screen but without badge earned.	Output: Results screen
	5	Proceed to Step 16 of base flow (results display).		
9-11			Condition: Player exits game before completion	
	1	Player	Clicks Exit option.	Input: Click action on Exit button
	2	System	Displays confirmation dialog: "Are you sure you want to exit? Your progress will be lost." with "Yes" and "No" buttons.	Output: Confirmation dialog
	3	Player	Clicks "Yes" to confirm exit.	Input: Click action on Yes button
	4	System	Redirects player to homepage.	Output: Homepage display

	5	Use case ends.		
--	---	----------------	--	--

UC-PG2: Play Whiz Memory Match

Use Case Name

Play Whiz Memory Match

Use Case ID

UC-PG2

Description

Goal

To allow a logged-in player to play the Whiz Memory Match game mode by flipping and matching pairs of cards, testing their memory skills while earning points based on performance.

Actions/Short Summary

- Player selects Whiz Memory Match game mode from homepage
- Player chooses difficulty level (Easy, Average, Difficult)
- System loads game board with cards based on difficulty
- Player flips cards to find matching pairs
- System validates matches and updates score
- Game ends when all pairs matched
- System displays results and updates player stats

Trigger

Player clicks on "Whiz Memory Match" game mode from the homepage because they want to test their memory skills through a card-matching challenge.

Actor(s)

1. **Primary Actor** - Player
2. **Secondary Actors** - System

Assumptions

1. The system is available and player session is active.
2. Card image/content database contains sufficient pairs for all difficulty levels.
3. Timer mechanism functions correctly and consistently.
4. Game board layout is appropriate for different difficulty levels.
5. Cards remain face-down initially and flip with animation when clicked.

- Score is calculated based on matches found, moves taken, and time taken.

Preconditions

- Player is logged into the system (has active session).
- Player has a valid account in the database.
- Card content database is populated with images/content for matching pairs.
- System can track and update player statistics.
- Player is on the homepage or game selection screen.

Post-conditions

- Player's game performance is recorded.
- Player statistics are updated in the database.
- Game results are displayed to player showing performance metrics.
- All matched pairs remain visible; unmatched cards return to face-down state.
- System is ready to accept next player action.

Scenario 1: Player Completes Game by Matching All Pairs

Base Flow

Step	Executor	Action/Description	Data (Input or Output)
1	Player	Clicks "Whiz Memory Match" button/card on the homepage.	Input: Click action on Whiz Memory Match
2	System	Displays difficulty level selection screen with available options.	Output: Difficulty selection screen (Easy, Average, Difficult buttons)
3	Player	Selects a difficulty level (Easy, Average, or Difficult) by clicking on it.	Input: Difficulty level selection (Easy OR Average OR Difficult)
4	System	Retrieves card content from database based on selected difficulty and generates random card pairs.	Input: Difficulty level; Output: Card pair set (card IDs, image URLs, positions)
5	System	Shuffles card positions randomly and initializes game board with all cards face-down.	Input: Card pair set; Output: Shuffled card arrangement
6	System	Displays game board with cards face-down, initializes moves to 0, and starts timer at 0:00 counting upward.	Output: Game screen
7	Player	Clicks on first card to flip it face-up.	Input: Click action on card position

8	System	Flips selected card face-up with animation and displays card content.	Input: Card position; Output: Card displayed face-up with content visible
9	Player	Clicks on second card to flip it face-up.	Input: Click action on different card position
10	System	Flips second card face-up and increments moves counter by 1.	Input: Card position; Output: Second card displayed face-up, moves counter updated
11	System	Compares the two flipped cards to check if they match.	Input: Card A content, Card B content; Output: Match result (TRUE or FALSE)
12	System	If cards match: Keeps both cards face-up permanently and increments moves. If cards don't match: Pauses for 1-2 seconds showing both cards, then flips both cards back to face-down.	Input: Match result; Output: Updated card states and moves count
13	System	Checks if all pairs have been matched.	Input: Number of matched pairs, total pairs; Output: Game complete status (FALSE - continue)
14	System	Repeats Steps 7-13 until all pairs are matched.	Input/Output: Card flips, match checks, moves count updates
15	System	Detects all pairs matched and stops timer.	Input: All pairs matched; Output: Timer stopped, final time recorded
16	System	Calculates moves taken (fewer = better), time taken (less = better).	Input: Moves, time elapsed; Output: Game results
17	System	Updates player statistics in database.	Input: Player ID, game results; Output: Updated player stats record
18	System	Displays results screen showing total moves and time taken.	Output: Results screen

Alternative Flows

Base Step	Alt Flow Step	Executor	Action/Description	Data (Input and/or Output)
-----------	---------------	----------	--------------------	----------------------------

7-14			Condition: Player exits game before completion	
	1	Player	Clicks Exit option.	Input: Click action on Exit button
	2	System	Displays confirmation dialog: "Are you sure you want to exit? Your progress will be lost." with "Yes" and "No" buttons.	Output: Confirmation dialog
	3	Player	Clicks "Yes" to confirm exit.	Input: Click action on Yes button
	4	System	Redirects player to homepage.	Output: Homepage display
	5	Use case ends.		

UC-PG3: Play Whiz Puzzle

Use Case Name

Play Whiz Puzzle

Use Case ID

UC-PG3

Description

Goal

To allow a logged-in player to play the Whiz Puzzle game mode by arranging scrambled image pieces to form a complete picture, testing their spatial reasoning and problem-solving skills.

Actions/Short Summary

- Player selects Whiz Puzzle game mode from homepage
- Player chooses difficulty level (Easy, Average, Difficult)
- Player chooses category (Solar System, Scientists, The Human Body, Animals, Geometry, Starbooks)
- System loads scrambled puzzle pieces based on selections
- Player drags and arranges pieces to form correct image
- System validates puzzle completion
- System displays results and updates player stats

Trigger

Player clicks on "Whiz Puzzle" game mode from the homepage because they want to test their puzzle-solving skills by completing image puzzles.

Actor(s)

1. **Primary Actor** - Player
2. **Secondary Actors** - System

Assumptions

1. The system is available and player session is active.
2. Image database contains sufficient puzzle images for all categories and difficulty levels.
3. Timer mechanism functions correctly to track time.
4. Puzzle piece counts vary by difficulty.
5. Pieces can be dragged and dropped.
6. System validates puzzle completion by checking all pieces are in correct positions.

Preconditions

1. Player is logged into the system (has active session).
2. Player has a valid account in the database.
3. Puzzle image database is populated with images for all categories.
4. System can track and update player statistics.
5. Player is on the homepage or game selection screen.

Post-conditions

1. Player's game performance is recorded.
2. Player statistics are updated in the database.
3. Game results are displayed to player.
4. Completed puzzle image is displayed in full view.
5. System is ready to accept next player action.

Scenario 1: Player Completes Puzzle Successfully**Base Flow**

Step	Executor	Action/Description	Data (Input or Output)
1	Player	Clicks "Whiz Puzzle" button/card on the homepage.	Input: Click action on Whiz Puzzle
2	System	Displays difficulty level selection screen with available options.	Output: Difficulty selection screen (Easy, Average, Difficult)

3	Player	Selects a difficulty level (Easy, Average, or Difficult) by clicking on it.	Input: Difficulty level selection (Easy OR Average OR Difficult)
4	System	Records selected difficulty and displays category selection screen.	Input: Difficulty from Step 3; Output: Category selection screen (Solar System, Scientists, The Human Body, Animals, Geometry, Starbooks buttons)
5	Player	Selects a category by clicking on it.	Input: Category selection (Solar System OR Scientists OR The Human Body OR Animals OR Geometry OR Starbooks)
6	System	Retrieves puzzle image from database based on selected category and difficulty.	Input: Category, Difficulty level; Output: Puzzle image data (image URL, correct piece arrangement)
7	System	Divides image into pieces based on difficulty level and scrambles their positions randomly.	Input: Puzzle image, difficulty level; Output: Scrambled puzzle pieces with position IDs
8	System	Displays puzzle board with scrambled pieces, initializes moves counter to 0, and starts timer at 0:00 counting upward.	Output: Game screen
9	Player	Drags a puzzle piece and drops it to swap with another piece OR clicks two pieces to swap their positions.	Input: Piece A position, Piece B position, swap action
10	System	Swaps the positions of the two selected pieces and increments moves counter by 1.	Input: Piece positions; Output: Updated puzzle board with swapped pieces, moves counter updated (e.g., "1")
11	System	Checks if all puzzle pieces are now in their correct positions.	Input: Current piece arrangement, correct piece arrangement; Output: Completion status (FALSE - continue)
12	System	Repeats Steps 9-11 until puzzle is complete.	Input/Output: Piece swaps, validation checks, moves updates
13	System	Detects all pieces are in correct positions and stops timer.	Input: All pieces correctly arranged; Output: Timer stopped, final time recorded
14	System	Displays completion animation (puzzle pieces lock in place, brief celebration effect).	Output: Completion animation

15	System	Calculates completion time (less = better) and moves taken (fewer = better).	Input: Time elapsed, moves taken, difficulty level; Output: Game results
16	System	Updates player statistics in database	Input: Player ID, game results; Output: Updated player stats record
17	System	Displays results screen	Output: Results screen

Alternative Flows

Base Step	Alt Flow Step	Executor	Action/Description	Data (Input and/or Output)
9-12			Condition: Player exits game before completion	
	1	Player	Clicks Exit option.	Input: Click action on Exit button
	2	System	Displays confirmation dialog: "Are you sure you want to exit? Your progress will be lost." with "Yes" and "No" buttons.	Output: Confirmation dialog
	3	Player	Clicks "Yes" to confirm exit.	Input: Click action on Yes button
	4	System	Redirects player to homepage.	Output: Homepage display
	5	Use case ends.		

UC-PG4: Play Whiz Battle**Use Case Name**

Play Whiz Battle

Use Case ID

UC-PG4

Description**Goal**

To allow a logged-in player to compete against another player in real-time by either creating a battle room or joining an existing one, answering questions simultaneously to determine a winner based on performance.

Actions/Short Summary

- Player selects Whiz Battle game mode from homepage
- Player chooses to either Create Battle or Join Battle
- If creating: Player selects category and difficulty, system generates battle room code
- If joining: Player enters battle room code
- Both players compete by answering same questions in real-time
- System calculates scores/stars for both players
- System determines winner and updates stats for both players

Trigger

Player clicks on "Whiz Battle" game mode from the homepage because they want to compete against another player in a real-time quiz battle.

Actor(s)

1. **Primary Actor** - Player
2. **Secondary Actors**
 - System
 - Opponent Player

Assumptions

1. The system is available and both players have active sessions.
2. Question bank contains sufficient questions for all categories and difficulty levels.
3. Battle room codes are unique and have expiration times.
4. System supports real-time synchronization between two players.
5. Winner is determined by number of correct answers; in case of tie, player who earned more stars due to faster submission of answers wins.
6. Both players must answer the same set of questions simultaneously.
7. Battle rooms can only accommodate exactly 2 players (1 creator + 1 joiner).

Preconditions

1. Player is logged into the system (has active session).
2. Player has a valid account in the database.
3. For Join Battle: A valid battle room exists with available slot.
4. Question database contains questions for selected category and difficulty.
5. System can track and update player statistics for both players.
6. Real-time communication infrastructure is operational.

Post-conditions

1. Battle match results are recorded for both players.
2. Player statistics are updated in the database for both players.
3. Battle room is closed after match completion.
4. Match results are displayed to both players.
5. System is ready to accept next player action.

Scenario 1: Player Creates Battle and Completes Match Successfully**Base Flow**

Step	Executor	Action/Description	Data (Input or Output)
1	Player	Clicks "Whiz Battle" button/card on the homepage.	Input: Click action on Whiz Battle
2	System	Displays battle mode selection screen with two options.	Output: Mode selection screen ("Create Battle" button, "Join Battle" button)
3	Player	Clicks "Create Battle" button.	Input: Click action on Create Battle
4	System	Displays category and difficulty selection screen.	Output: Selection screen (Science/Math category buttons, Easy/Average/Difficult difficulty buttons)
5	Player	Selects a category (Science or Math).	Input: Category selection (Science OR Math)
6	Player	Selects a difficulty level (Easy, Average, or Difficult).	Input: Difficulty level selection (Easy OR Average OR Difficult)
7	System	Records selections and retrieves question set from database based on category and difficulty.	Input: Category, Difficulty; Output: Question set (questions with IDs, text, options, correct answers)
8	System	Creates new battle room with unique room ID and generates shareable 6-digit game code.	Input: Creator Player ID, category, difficulty, question set; Output: Battle room ID, game code
9	System	Displays waiting room screen with game code prominently displayed and "Copy Code" button.	Output: Waiting room (game code display, room status, "Cancel Battle" button)
10	Player	Shares game code with opponent	Input: Game code shared externally
11	Opponent Player	Enters the game code in their Join Battle screen and clicks "Join Battle" button.	Input: Game code entry by opponent

12	System	Validates game code, verifies room has available slot, and adds opponent to the battle room.	Input: Game code, Opponent Player ID; Output: Battle room updated with both players
13	System	Notifies creator player that opponent has joined and displays both player names/avatars.	Output: Updated waiting room (both player profiles shown, "Opponent joined!" message, "Start Battle" button enabled)
14	Player (Creator)	Clicks "Start Battle" button when ready.	Input: Click action on Start Battle
15	System	Begins countdown and simultaneously loads first question for both players.	Output: Countdown animation, then game screen with first question for both players
16	System	Starts timer and displays question with answer options to both players simultaneously.	Output: Battle game screen (question counter "1/10", question text, answer options, timer)
17	Both Players	Read question and select their answer	Input: Answer selections from both players
18	System	Records each answer with timestamp as soon as submitted.	Input: Answer, timestamp; Output: Answer recorded
19	System	When both players submit, evaluates both answers (correct/incorrect) and awards stars.	Input: Both player answers, correct answer; Output: Correctness result for both, stars awarded
20	System	Displays brief result feedback showing both players' answers and current score comparison.	Output: Answer results and current star count for both
21	System	Loads next question and repeats Steps 16-20 until all questions are answered.	Input/Output: Question display, answer submissions, scoring
22	System	Calculates final scores and stars for both players.	Input: All answers from both players; Output: Final scores
23	System	Determines winner: Player with more stars wins; if tied, player with faster total completion time wins.	Input: Both final scores, total times; Output: Winner determination (Player 1 OR Player 2 OR Tie)
24	System	Awards badge for the completed difficulty level of the winner.	Input: Category, Difficulty level, Player ID; Output: Badge record
25	System	Updates statistics for both players.	Input: Battle results, both Player IDs; Output: Updated stats records for both players
26	System	Closes the battle room.	Input: Battle room ID; Output: Room status changed to "closed"

27	System	Displays results screen to both players showing winner announcement, final scores, and badge earned (for winner).	Output: Results screen
----	--------	---	------------------------

Scenario 2: Player Joins Existing Battle**Base Flow**

Step	Executor	Action/Description	Data (Input or Output)
1-2			Same as Scenario 1, Steps 1-2
3	Player	Clicks "Join Battle" button.	Input: Click action on Join Battle
4	System	Displays game code entry screen.	Output: Code entry screen (6-digit code input field, "Join Battle" button)
5	Player	Enters 6-digit game code received from battle creator and clicks "Join Battle" button.	Input: Game code
6	System	Validates game code format and checks if battle room exists and has available slot.	Input: Game code; Output: Battle room data
7	System	Adds player to the battle room as opponent.	Input: Player ID, Battle room ID; Output: Updated battle room with both players
8	System	Notifies creator that opponent has joined and displays waiting screen to joiner.	Output: Joiner sees "Joined successfully! Waiting for host to start..." message with both player profiles
9-27			Continue with Scenario 1 Steps 14-26 (from "Start Battle" onwards)

Alternative Flows

Base Step	Alt Flow Step	Executor	Action/Description	Data (Input and/or Output)
6 (Scenario 2)			Condition: Invalid or expired game code entered	
	1	System	Validates game code and finds no matching active battle room.	Input: Game code; Output: No match found or room expired

	2	System	Displays error message: "Invalid code. Please check and try again."	Output: Error message
	3	System	Returns to code entry screen with input field cleared.	Output: Code entry screen
	4	Resume Scenario 2 at Step 5 (player can try entering code again).		
17-21			Condition: Opponent disconnects or exits mid-game	
	1	Opponent Player	Loses connection or clicks exit button during active battle.	Input: Disconnect event or exit action
	2	System	Detects opponent disconnection/exit and immediately ends the match.	Input: Connection status; Output: Match terminated
	3	System	Automatically declares remaining player as winner by forfeit.	Input: Remaining player ID; Output: Winner by forfeit
	4	System	Updates statistics: remaining player gets win, disconnected opponent gets loss.	Input: Both Player IDs, forfeit result; Output: Updated stats (win for one, loss for other)
	5	System	Displays victory screen to remaining player	Output: Forfeit victory screen
	6	System	Displays disconnection screen to opponent (if they reconnect): "Connection lost. Match forfeited."	Output: Forfeit screen
	7	Proceed to Step 25 of Scenario 1 (close room and show results).		
17-21			Condition: Primary player exits mid-game	
	1	Player	Clicks exit button or loses connection during active battle.	Input: Exit action or disconnect event

	2	System	Displays confirmation dialog: "Exiting will forfeit the match. Continue?"	Output: Confirmation dialog
	3	Player	Clicks "Yes" to confirm exit.	Input: Click action on Yes
	4	System	Ends match and declares opponent as winner by forfeit.	Input: Opponent Player ID; Output: Winner by forfeit
	5	System	Updates statistics: opponent gets win, exiting player gets loss.	Input: Both Player IDs, forfeit result; Output: Updated stats
	6	System	Displays forfeit screen to exiting player: "You forfeited. Opponent wins."	Output: Forfeit screen
	7	System	Displays forfeit victory screen to opponent.	Output: Victory screen
	8	Proceed to Step 25 of Scenario 1 (close room).		

UC-RR1: Player Requests Reward

Use Case Name

Player Requests Reward

Use Case ID

UC-RR1

Description

Goal

To allow a logged-in player who has earned all required badges in a difficulty level to submit a reward redemption request, which will be processed by an admin for verification and approval.

Actions/Short Summary

- Player navigates to badge collection page
- System displays earned badges and reward eligibility status
- Player selects to claim reward for eligible difficulty level
- System validates badge requirements are met
- System creates reward request record
- System confirms request submission to player

Trigger

Player clicks "Your Badges" from the profile menu because they have completed all badge requirements for a difficulty level and want to redeem their reward.

Actor(s)

1. **Primary Actor** - Player
2. **Secondary Actors** - System

Assumptions

1. The system is available and player session is active.
2. Each difficulty level (Easy, Average, Difficult) has exactly 3 badges to earn.
3. Reward eligibility requires all 3 badges within the same difficulty level.
4. Players can have multiple pending reward requests for different difficulty levels simultaneously.

Preconditions

1. Player is logged into the system (has active session).
2. Player has a valid account in the database.
3. Player has earned at least some badges (may or may not be eligible for rewards yet).
4. Badge collection data is accessible and up-to-date.
5. Admin system for processing reward requests is operational.

Post-conditions

1. Player's reward request is created and stored in database with "Pending" status.
2. Request is added to admin's reward processing queue.
3. Reward request is sent to admin system.
4. System is ready to accept additional player actions.

Scenario 1: Player Successfully Requests Reward for Completed Difficulty Level**Base Flow**

Step	Executor	Action/Description	Data (Input or Output)
1	Player	Clicks "Your Badges" link from profile menu or homepage.	Input: Click action on Your Badges
2	System	Retrieves player's badge collection data from database.	Input: Player ID; Output: Badge records (badge IDs, difficulty levels)
3	System	Analyzes badge collection to determine reward eligibility for each difficulty level (3 badges = eligible).	Input: Badge collection; Output: Eligibility status per difficulty

4	System	Checks for any existing pending or approved reward requests for each difficulty level.	Input: Player ID; Output: Existing reward request records with status
5	System	Displays badge collection page showing all earned badges organized by difficulty level, with claim status buttons.	Output: Badge collection page (badges displayed by difficulty, earned badges, empty slots for unearned badges, "Claim" button for eligible levels, "Locked" button for ineligible levels)
6	Player	Reviews badge collection and identifies eligible difficulty level with "Claim" button enabled.	Input: Visual review
7	Player	Clicks "Claim" button for chosen difficulty level.	Input: Click action on Claim button, difficulty level selection
8	System	Creates new reward request record in database.	Input: Player ID, difficulty level, request timestamp, badge IDs; Output: Reward request record (request ID, player ID, difficulty level, status: "Pending", timestamp)

Alternative Flows

Base Step	Alt Flow Step	Executor	Action/Description	Data (Input and/or Output)
2-5			Condition: Player has no badges earned yet	
	1	System	Retrieves badge collection and finds zero earned badges.	Input: Player ID; Output: Empty badge collection
	2	System	Displays badge collection page with all badge slots empty and all claim buttons locked.	Output: Badge collection page (empty badge slots with placeholders, all difficulty levels show "Locked" buttons)
	3	Use case ends (player cannot proceed with reward request).		
5			Condition: Player has some badges but not eligible for any rewards yet	

	1	System	Displays badge collection showing earned badges but all claim buttons remain locked.	Output: Badge collection page (partial badges displayed, "Locked" buttons for all difficulty levels)
	2	Use case ends (player cannot claim reward yet).		

UC-RR2: Admin Grants Reward

Use Case Name

Admin Grants Reward

Use Case ID

UC-RR2

Description

Goal

To allow an admin to review, verify, and process player reward requests by granting rewards to eligible players who have earned all required badges in a difficulty level, updating the system accordingly.

Actions/Short Summary

- Admin navigates to player management section
- Admin selects player profile to review
- Admin views player's badge collection and reward eligibility
- Admin verifies badge requirements are met
- Admin grants reward by clicking reward button
- System resets badges for redeemed difficulty level
- System logs reward transaction
- System updates player profile with reward claim record

Trigger

Admin accesses the admin panel and navigates to player management because they need to verify a player's badge collection and grant eligible rewards.

Actor(s)

1. **Primary Actor** - Admin
2. **Secondary Actors** - System

Assumptions

1. The system is available and admin session is active.
2. Reward granting resets the badge collection for that specific difficulty level.
3. Each difficulty level can be redeemed multiple times.
4. System maintains audit logs of all reward transactions for accountability.
5. Physical rewards (if applicable) are handled outside the system.

Preconditions

1. Admin is logged into the admin panel (has active admin session).
2. Admin has valid admin account.
3. At least one player has submitted a reward request OR has eligible badges.
4. Player profile data and badge collection are accessible in the system.
5. Reward transaction logging system is operational.

Post-conditions

1. Reward claim is marked as "Approved" in the system.
2. Player's badge collection for redeemed difficulty level is reset to zero badges.
3. Reward transaction is logged with timestamp, admin ID, player ID, and difficulty level.
4. Player profile is updated to show reward claim history.
5. System is ready for admin to process next reward request.

Scenario 1: Admin Successfully Grants Reward to Eligible Player**Base Flow**

Step	Executor	Action/Description	Data (Input or Output)
1	Admin	Logs into admin panel and navigates to "List of Players" section.	Input: Click action on List of Players menu item
2	System	Retrieves and displays list of all players with relevant information.	Output: Player list table showing badges earned count with "View" action button per row
3	Admin	Locates player with pending reward request or specific player to review, and clicks "View" button in the badges column.	Input: Click action on View button for selected player row
4	System	Retrieves selected player's complete badge collection data from database.	Input: Player ID; Output: Badge collection

5	System	Analyzes badge collection to determine reward eligibility for each difficulty level (3 badges earned = eligible).	Input: Badge collection; Output: Eligibility status per difficulty (Easy: TRUE/FALSE, Average: TRUE/FALSE, Difficult: TRUE/FALSE)
6	System	Retrieves any pending reward requests for this player.	Input: Player ID; Output: Reward request records with status
7	System	Displays player's badge collection page in admin view showing badges organized by difficulty level with reward action buttons.	Output: Admin badge view (badges displayed by difficulty level, "Reward" button enabled for eligible levels, "Locked" button for ineligible levels)
8	Admin	Reviews badge collection, verifies that player has earned all 3 badges for the selected difficulty level.	Input: Visual verification of badges
9	Admin	Clicks "Reward" button for the chosen eligible difficulty level.	Input: Click action on Reward button, difficulty level selection
10	System	Displays reward confirmation dialog with player and reward details.	Output: Confirmation dialog, badge list to be reset, "Confirm" button, "Cancel" button, optional notes field)
11	Admin	Reviews confirmation details and clicks "Confirm" button to proceed.	Input: Click action on Confirm button, optional notes text
12	System	Creates reward transaction record in database.	Input: Player ID, Admin ID, difficulty level, timestamp, badge IDs being redeemed; Output: Reward transaction record (transaction ID, all details, status: "Completed")
13	System	Resets player's badge collection for the redeemed difficulty level (removes all 3 badges for that level).	Input: Player ID, difficulty level; Output: Updated badge collection (badges for that difficulty removed, other difficulty badges preserved)
14	System	Refreshes badge collection view showing updated badge status	Output: Updated admin badge view (badges reset for redeemed level, other levels unchanged)

Alternative Flows

Base Step	Alt Flow Step	Executor	Action/Description	Data (Input and/or Output)
7			Condition: Player has no eligible rewards (all reward buttons locked)	

	1	System	Analyzes badge collection and determines no difficulty level has all 3 badges.	Input: Badge collection; Output: All eligibility statuses FALSE
	2	System	Displays badge collection with all "Reward" buttons shown as "Locked".	Output: Admin badge view (badges displayed, all reward buttons locked/disabled)
	3	Admin	Reviews page and confirms player is not yet eligible.	Input: Visual review
	4	Admin	Clicks "Back to List" to return to player list.	Input: Click action on Back to List
	5	System	Returns admin to player list page.	Output: Player list table
	6	Use case ends.		
11			Condition: Admin cancels reward action	
	1	Admin	Reviews confirmation dialog but decides not to proceed, clicks "Cancel" button.	Input: Click action on Cancel button
	2	System	Closes confirmation dialog without making any changes.	Input: None
	3	System	Returns admin to badge collection view (unchanged).	Output: Admin badge view with Reward button still available
	4	Use case ends (no changes made to player profile or badges).		

UC-MAA1: Admin Login

Use Case Name

Admin Login

Use Case ID

UC-MAA1

Description

Goal

To allow a registered admin to authenticate and access the STARBOOKS admin panel using their credentials, enabling them to manage the system and perform administrative functions.

Actions/Short Summary

- System displays admin login form
- Admin enters username and password
- System verifies credentials against admin database
- System authenticates admin and establishes session
- System redirects admin to admin dashboard

Trigger

Admin navigates to the admin login page because they need to access the admin panel to perform administrative tasks such as managing players, processing rewards, or configuring system settings.

Actor(s)

1. **Primary Actor** - Admin
2. **Secondary Actors** - System

Assumptions

1. The system is available and accessible.
2. The admin has already been added to the admin list.
3. Admin credentials are stored securely in a separate admin database table.
4. The system has a secure authentication mechanism for admin access.
5. Admin login is separate from player login.

Preconditions

1. System is available and operational.
2. Admin account exists in the admin database.
3. Admin is not currently logged in.
4. Admin login page is accessible.

Post-conditions

1. Admin is authenticated and logged into the admin panel.
2. Admin is redirected to the admin dashboard.
3. System is ready to process admin actions and requests.

Scenario 1: Successful Admin Login with Valid Credentials**Base Flow**

Step	Executor	Action/Description	Data (Input or Output)
------	----------	--------------------	------------------------

1	System	Displays admin login form with username and password fields.	Output: Admin login form (username field, password field, "Login" button)
2	Admin	Enters username and password into the respective fields and clicks "Login" button.	Input: admin username, admin password
3	System	Retrieves the entered credentials and validates format (fields not empty, proper data types).	Input: username, password from Step 2
4	System	Queries admin database to verify credentials match an existing admin account.	Input: username, password; Output: Admin account record (Admin ID, username, hashed password)
5	System	Compares entered password with stored hashed password.	Input: Entered password, stored hashed password
6	System	Redirects admin to the admin dashboard	Output: Admin dashboard display

Alternative Flows

Base Step	Alt Flow Step	Executor	Action/Description	Data (Input and/or Output)
4			Condition: Invalid credentials (username not found or password incorrect)	
	1	System	Queries admin database and finds no matching username OR password does not match.	Input: username, password; Output: No match found or password mismatch
	2	System	Displays error message: "Invalid credentials. Please try again."	Output: Error message (does not specify which field is incorrect for security reasons)
	3	System	Returns to Step 2 of base flow.	Output: Admin login form with empty fields
3			Condition: Empty login fields	
	1	System	Validates that username and password fields are not empty.	Input: username field, password field

	2	System	Displays error message: "Please enter both username and password."	Output: Error message
	3	System	Returns to Step 2 of base flow.	Output: Admin login form

UC-MAA2: Add New Admin

Use Case Name

Add New Admin

Use Case ID

UC-MAA2

Description

Goal

To allow a logged-in admin to create a new administrator account in the STARBOOKS system, enabling the new admin to access and manage the admin panel.

Actions/Short Summary

- Admin navigates to admin management section
- Admin initiates new admin creation
- System displays admin registration form
- Admin enters new admin details
- System validates the input data
- System creates new admin account
- System confirms successful account creation

Trigger

Admin clicks "Add New Admin" button from the List of Admins page because they need to grant administrative access to a new team member or staff.

Actor(s)

1. **Primary Actor** - Admin
2. **Secondary Actors** - System

Assumptions

1. The system is available and admin session is active.
2. All admin registration fields have defined validation rules.
3. Admin passwords are stored securely using encryption/hashing.

4. Username must be unique across all admin accounts.

Preconditions

1. Admin is logged into the admin panel (has active admin session).
2. Admin is on the List of Admins page or admin management section.
3. System can access admin database to create new records.

Post-conditions

1. A new admin account is created and stored in the admin database.
2. New admin account is added to the List of Admins.
3. New admin can now use their credentials to login.
4. System is ready to accept additional admin actions.

Scenario 1: Admin Successfully Creates New Admin Account

Base Flow

Step	Executor	Action/Description	Data (Input or Output)
1	Admin	Navigates to admin panel and clicks on "List of Admins" menu item.	Input: Click action on List of Admins
2	System	Retrieves and displays list of all existing admin accounts.	Output: Admin list table
3	Admin	Clicks "Add New Admin" button.	Input: Click action on Add New Admin
4	System	Displays admin registration form with required fields.	Output: Admin registration form (username field, password field, confirm password field, sex dropdown button, image upload field, "Submit" button, "Cancel" button)
5	Admin	Enters all required details for the new admin account: username, password, confirms password, selects sex, optionally uploads profile image, then clicks "Submit" button.	Input: new admin username, new admin password, confirm password, sex selection, profile image file (optional)
6	System	Validates all input fields: checks that all required fields are filled, username format is valid, password meets security requirements, passwords match, sex is selected, image format is valid (if uploaded).	Input: All form data from Step 5

7	System	Checks username uniqueness by querying admin database to ensure username does not already exist.	Input: New admin username; Output: Username availability check (AVAILABLE)
8	System	Processes profile image (if uploaded): validates file type (jpg, png), resizes if needed, stores in image storage.	Input: Image file; Output: Image URL or stored image path
9	System	Hashes the new admin password using secure hashing algorithm.	Input: New admin password (plain text); Output: Hashed password
10	System	Creates new admin account record in admin database.	Input: Username, hashed password, sex, image URL, creator Admin ID; Output: New admin record
11	System	Displays success message: "New admin account '[Username]' has been created successfully!"	Output: Success message with new admin username
12	System	Redirects admin back to List of Admins page showing updated list with new admin included.	Output: Updated admin list table with new admin entry

Alternative Flows

Base Step	Alt Flow Step	Executor	Action/Description	Data (Input and/or Output)
6			Condition: Required fields are empty or invalid	
	1	System	Identifies missing required fields or invalid formats.	Input: Incomplete or invalid form data
	2	System	Displays error message: "Please fill in all required fields" or field-specific error messages (e.g., "Invalid username format") and highlights problematic fields in red.	Output: Error message, field highlighting
	3	System	Returns to Step 5 of base flow.	Output: Admin registration form with existing data preserved
7			Condition: Username already exists	
	1	System	Queries admin database and finds username already in use.	Input: New admin username; Output:

				Username already exists
	2	System	Displays error message: "Username '[Username]' already exists. Please choose a different username."	Output: Error message with username highlighted
	3	System	Returns to Step 5 of base flow.	Output: Admin registration form with data preserved
6			Condition: Password does not meet security requirements	
	1	System	Validates password against security rules (minimum 8 characters, complexity requirements).	Input: New admin password; Output: Validation result (FAIL)
	2	System	Displays error message: "Password must be at least 8 characters and include uppercase, lowercase, and numbers."	Output: Error message with password requirements
	3	System	Returns to Step 5 of base flow.	Output: Admin registration form with data preserved

UC-MAA3: Edit Admin Profile

Use Case Name

Edit Admin Profile

Use Case ID

UC-MAA3

Description

Goal

To allow a logged-in admin to update another administrator's profile information in the STARBOOKS system, ensuring admin account details remain current and accurate.

Actions/Short Summary

- Admin navigates to admin management section
- Admin selects specific admin profile to edit
- System displays current admin information in editable form
- Admin modifies desired fields

- System validates the updated information
- System saves changes to database
- System confirms successful profile update

Trigger

Admin clicks the "Edit" icon next to an admin entry in the List of Admins because they need to update that administrator's profile information.

Actor(s)

1. **Primary Actor** - Admin
2. **Secondary Actors** - System

Assumptions

1. The system is available and admin session is active.
2. All profile fields have defined validation rules.
3. Admins can edit other admin profiles, including their own.

Preconditions

1. Admin is logged into the admin panel (has active admin session).
2. Target admin account exists in the database.
3. Admin is on the List of Admins page.
4. System can retrieve current admin profile data.

Post-conditions

1. Selected admin profile information is updated in the database.
2. System displays confirmation message to editing admin.
3. Updated profile information is reflected in the List of Admins.
4. System is ready to accept additional admin actions.

Scenario 1: Admin Successfully Updates Another Admin's Profile**Base Flow**

Step	Executor	Action/Description	Data (Input or Output)
1	Admin	Navigates to "List of Admins" from the admin panel menu.	Input: Click action on List of Admins
2	System	Retrieves and displays list of all existing admin accounts.	Output: Admin list table
3	Admin	Locates the target admin profile to edit and clicks the "Edit" icon for that row.	Input: Click action on Edit icon for selected admin

4	System	Retrieves selected admin's current profile data from database.	Input: Target Admin ID; Output: Admin profile data (username, sex, profile image URL, registration date)
5	System	Displays Edit Admin Profile form with fields pre-populated with current information.	Output: Edit admin form (username field, sex dropdown pre-selected, current profile image displayed with "Change Image" option, "Save Changes" button, "Cancel" button)
6	Admin	Modifies one or more editable fields and clicks "Save Changes" button.	Input: Updated values for modified fields
7	System	Validates all input fields: checks that required fields are not empty, sex selection is valid, image format is valid (if changed).	Input: Updated profile data from Step 6
8	System	If image was changed: processes new profile image (validates file type, resizes if needed, stores in image storage, deletes old image if applicable).	Input: New image file (if provided); Output: New image URL or stored image path
9	System	Compares new data with existing data and identifies changed fields.	Input: New profile data, existing profile data; Output: List of changed fields
10	System	Updates admin profile record in database with new information.	Input: Target Admin ID, updated field values; Output: Updated admin profile record
11	System	Displays success message: "Admin profile '[Username]' has been updated successfully!"	Output: Success message with admin username
12	System	Redirects admin back to List of Admins page showing updated information.	Output: Updated admin list table with modified profile data displayed

Alternative Flows

Base Step	Alt Flow Step	Executor	Action/Description	Data (Input and/or Output)
7			Condition: Invalid or empty required fields	
	1	System	Identifies required fields that are empty or have invalid values.	Input: Profile form data from Step 6

	2	System	Displays validation error message: "Please correct the following errors:" followed by field-specific messages and highlights invalid fields in red.	Output: Error message, field highlighting
	3	System	Returns to Step 6 of base flow.	Output: Edit admin form with entered data preserved

UC-MAA4: Change Admin Password

Use Case Name

Change Admin Password

Use Case ID

UC-MAA4

Description

Goal

To allow a logged-in admin to change an administrator's account password in the STARBOOKS system, ensuring account security and allowing password updates when needed.

Actions/Short Summary

- Admin navigates to admin management section
- Admin selects specific admin account for password change
- System displays password change form
- Admin enters current password and new password
- System verifies current password is correct
- System validates new password meets security requirements
- System updates password in database
- System confirms successful password change

Trigger

Admin clicks the "Password" icon next to an admin entry in the List of Admins because they need to change that administrator's password for security reasons.

Actor(s)

1. **Primary Actor** - Admin
2. **Secondary Actors** - System

Assumptions

1. The system is available and admin session is active.

2. Passwords are stored securely using encryption/hashing in the database.
3. The system has defined password security requirements (minimum length, complexity rules).
4. Admin must know the current password of the target account to change it.
5. Password change requires immediate effect.

Preconditions

1. Admin is logged into the admin panel (has active admin session).
2. Target admin account exists in the database with an existing password.
3. Admin is on the List of Admins page.
4. Change password form/dialog is accessible.

Post-conditions

1. Target admin's password is updated in the database with new hashed password.
2. Old password is replaced and can no longer be used for login.
3. System displays confirmation message to editing admin.
4. Target admin session remains active.
5. System is ready to accept additional admin actions.

Scenario 1: Admin Successfully Changes Another Admin's Password

Base Flow

Step	Executor	Action/Description	Data (Input or Output)
1	Admin	Navigates to "List of Admins" from the admin panel menu.	Input: Click action on List of Admins
2	System	Retrieves and displays list of all existing admin accounts.	Output: Admin list table
3	Admin	Locates the target admin account and clicks the "Password" icon for that row.	Input: Click action on Password icon for selected admin
4	System	Displays Change Password dialog/form with input fields.	Output: Change password form (target admin username displayed, current password field, new password field, confirm new password field, "Update Password" button, "Cancel" button)
5	Admin	Enters current password, new password, and confirms new password by re-entering it. Clicks "Update Password" button.	Input: current password, new password, confirm new password

6	System	Validates that all three fields are not empty.	Input: Form data from Step 5
7	System	Retrieves target admin's stored hashed password from database.	Input: Target Admin ID; Output: Stored hashed password
8	System	Verifies entered current password matches the stored hashed password.	Input: Entered current password, stored hashed password
9	System	Validates new password matches confirm new password field.	Input: New password, confirm new password
10	System	Validates new password meets security requirements (minimum 8 characters, includes uppercase, lowercase, and numbers).	Input: New password; Output: Validation result (PASS)
11	System	Checks that new password is different from current password.	Input: New password, current password; Output: Different passwords confirmed
12	System	Hashes the new password using secure hashing algorithm.	Input: New password (plain text); Output: Hashed new password
13	System	Updates target admin's password in database with the new hashed password.	Input: Target Admin ID, new hashed password; Output: Updated password record
14	System	Displays success message: "Password for '[Username]' has been changed successfully!"	Output: Success message
15	System	Closes the Change Password dialog and returns admin to List of Admins page.	Output: Admin list table

Alternative Flows

Base Step	Alt Flow Step	Executor	Action/Description	Data (Input and/or Output)
6			Condition: Empty password fields	
	1	System	Identifies one or more password fields are empty.	Input: Form data from Step 5
	2	System	Displays error message: "Please fill in all password fields."	Output: Error message
	3	System	Returns to Step 5 of base flow.	Output: Change password form

8			Condition: Incorrect current password	
	1	System	Compares entered current password with stored hashed password and finds no match.	Input: Entered current password, stored hashed password
	2	System	Displays error message: "Current password is incorrect. Please try again."	Output: Error message
	3	System	Clears all password fields for security.	Output: Change password form with empty fields
	4	System	Returns to Step 5 of base flow.	Output: Change password form
9			Condition: New password and confirm password do not match	
	1	System	Compares new password field with confirm new password field and finds mismatch.	Input: New password, confirm new password
	2	System	Displays error message: "New passwords do not match. Please re-enter."	Output: Error message
	3	System	Clears new password and confirm new password fields.	Output: Change password form with current password preserved
	4	System	Returns to Step 5 of base flow.	Output: Change password form
10			Condition: New password does not meet security requirements	
	1	System	Validates new password against security rules and identifies violations.	Input: New password
	2	System	Displays error message: "Password must be at least 8 characters and include uppercase, lowercase, and numbers."	Output: Error message with specific requirements
	3	System	Clears new password and confirm new password fields.	Output: Change password form with current password preserved
	4	System	Returns to Step 5 of base flow.	Output: Change password form

11			Condition: New password is same as current password	
	1	System	Compares new password with current password and finds they are identical.	Input: New password, current password
	2	System	Displays error message: "New password must be different from current password."	Output: Error message
	3	System	Clears new password and confirm new password fields.	Output: Change password form with current password preserved
	4	System	Returns to Step 5 of base flow.	Output: Change password form

UC-MAA5: Delete Admin

Use Case Name

Delete Admin

Use Case ID

UC-MAA5

Description

Goal

To allow a logged-in admin to remove an administrator account from the STARBOOKS system, revoking that admin's access to the admin panel.

Actions/Short Summary

- Admin navigates to admin management section
- Admin selects specific admin account to delete
- System displays confirmation dialog
- Admin confirms deletion intention
- System validates deletion is allowed
- System removes admin account from database
- System updates admin list display

Trigger

Admin clicks the "Delete" icon next to an admin entry in the List of Admins because that administrator account is no longer needed, the person has left the organization, or access needs to be revoked.

Actor(s)

1. **Primary Actor** - Admin
2. **Secondary Actors** - System

Assumptions

1. The system is available and admin session is active.
2. Admins cannot delete their own account while logged in.
3. Deletion is permanent and cannot be easily undone.
4. At least one admin account must remain in the system.

Preconditions

1. Admin is logged into the admin panel (has active admin session).
2. Target admin account exists in the database.
3. Target admin is not the currently logged-in admin.
4. Admin is on the List of Admins page.
5. System has at least 2 admin accounts.

Post-conditions

1. Selected admin account is removed from the admin database.
2. Deleted admin can no longer login to the system.
3. If target admin was logged in, their session is terminated.
4. Admin list is updated to reflect the deletion.
5. System is ready to accept additional admin actions.

Scenario 1: Admin Successfully Deletes Another Admin Account**Base Flow**

Step	Executor	Action/Description	Data (Input or Output)
1	Admin	Navigates to "List of Admins" from the admin panel menu.	Input: Click action on List of Admins
2	System	Retrieves and displays list of all existing admin accounts.	Output: Admin list table (username, sex, "Edit" icon, "Password" icon, "Delete" icon per row)
3	Admin	Locates the target admin account to delete and clicks the "Delete" icon for that row.	Input: Click action on Delete icon for selected admin
4	System	Validates that target admin is not the currently logged-in admin.	Input: Target Admin ID, current logged-in Admin ID; Output: Validation result (PASS - different admins)

5	System	Counts total number of active admin accounts to ensure at least one will remain after deletion.	Input: Active admin count; Output: Count validation (≥ 2 admins)
6	System	Displays deletion confirmation dialog with warning message.	Output: Confirmation dialog ("Are you sure you want to delete admin '[Username]'? This action cannot be undone.", target admin username and details displayed, "Confirm Delete" button, "Cancel" button)
7	Admin	Reviews the confirmation dialog and target admin details, then clicks "Confirm Delete" button.	Input: Click action on Confirm Delete button
8	System	Retrieves target admin's complete account data for archival.	Input: Target Admin ID; Output: Complete admin record (all fields, activity history)
9	System	If target admin is currently logged in elsewhere: terminates all active sessions for that admin.	Input: Target Admin ID; Output: Sessions terminated, session tokens invalidated
10	System	Displays success message: "Admin account '[Username]' has been deleted successfully."	Output: Success message with deleted admin username
11	System	Refreshes List of Admins page showing updated list without the deleted admin.	Output: Updated admin list table (deleted admin removed from display)

Alternative Flows

Base Step	Alt Flow Step	Executor	Action/Description	Data (Input and/or Output)
7			Condition: Admin cancels deletion	
	1	Admin	Reviews confirmation dialog and clicks "Cancel" button instead of "Confirm Delete".	Input: Click action on Cancel button
	2	System	Closes the confirmation dialog without making any changes.	Input: None
	3	System	Returns admin to List of Admins page with all accounts unchanged.	Output: Admin list table unchanged

	4	Use case ends (no deletion performed).		
4			Condition: Admin attempts to delete their own account	
	1	System	Validates target admin against current logged-in admin and detects they are the same.	Input: Target Admin ID, current Admin ID; Output: Self-deletion attempt detected
	2	System	Displays error message: "You cannot delete your own account while logged in. Please ask another administrator to remove your account."	Output: Error message
	3	System	Does not display confirmation dialog.	
	4	System	Returns admin to List of Admins page.	Output: Admin list table unchanged
	5	Use case ends (deletion prevented).		

UC-MAA6: Admin Logout

Use Case Name

Admin Logout

Use Case ID

UC-MAA6

Description

Goal

To allow a logged-in admin to safely end their session and exit the STARBOOKS admin panel, ensuring account security by terminating access and clearing session data.

Actions/Short Summary

- Admin initiates logout action
- System terminates admin session
- System redirects admin to login page

Trigger

Admin clicks on their avatar/profile icon and selects "Logout" from the dropdown menu because they want to end their admin session.

Actor(s)

1. **Primary Actor** - Admin
2. **Secondary Actors** - System

Assumptions

1. The system is available and admin session is active.
2. The system maintains admin session data that can be terminated.
3. Logout is accessible from all pages within the admin panel via avatar menu.

Preconditions

1. Admin is logged into the admin panel (has active admin session).
2. Admin session is valid and not expired.
3. Logout option is accessible from the avatar/profile dropdown menu.
4. Admin is on any page within the admin panel.

Post-conditions

1. Admin session is terminated and removed from active sessions.
2. Admin is logged out of the admin panel.
3. Admin is redirected to the admin login page.
4. System no longer recognizes admin as authenticated.
5. Admin must login again to access the admin panel.

Scenario 1: Admin Successfully Logs Out**Base Flow**

Step	Executor	Action/Description	Data (Input or Output)
1	Admin	Clicks on avatar/profile icon to open dropdown menu.	Input: Click action on avatar
2	System	Displays dropdown menu with account options including "Logout" option.	Output: Dropdown menu
3	Admin	Clicks "Logout" option from the dropdown menu.	Input: Click action on Logout
4	System	Displays confirmation dialog: "Are you sure you want to logout?" with "Yes" and "No" buttons.	Output: Confirmation dialog (message, Yes button, No button)
5	Admin	Clicks "Yes" button to confirm logout.	Input: Click action on Yes button
6	System	Displays logout success message: "You have been logged out successfully."	Output: Success message

7	System	Redirects admin to the login page.	Output: Login page display
---	--------	------------------------------------	----------------------------

Alternative Flows

Base Step	Alt Flow Step	Executor	Action/Description	Data (Input and/or Output)
5			Condition: Admin cancels logout	
	1	Admin	Clicks "No" button or closes the confirmation dialog.	Input: Click action on No button or dialog close
	2	System	Closes the confirmation dialog without taking any action.	Input: None
	3	System	Returns admin to the previous page they were viewing.	Output: Previous page display with admin still logged in
	4	Admin session remains active and valid.	Output: Active session maintained	
	5	Use case ends.		

UC-MQS1: Add New Question**Use Case Name**

Add New Question

Use Case ID

UC-MQS1

Description**Goal**

To allow a logged-in admin to add a new quiz question to the question bank in the STARBOOKS system, expanding the available content across categories and difficulty levels.

Actions/Short Summary

- Admin navigates to question management section
- Admin initiates new question creation
- System displays question entry form
- Admin enters question details

- System validates the input data
- System saves new question to question bank
- System confirms successful addition and updates question list

Trigger

Admin clicks "Add New Question" button from the Questions management page because they need to expand the question bank with new content for Whiz Challenge and Whiz Battle.

Actor(s)

1. **Primary Actor** - Admin
2. **Secondary Actors** - System

Assumptions

1. The system is available and admin session is active.
2. Question bank exists and is accessible in the database.
3. Categories include Science and Math.
4. Year levels represent educational grades.
5. Difficulty levels include: Easy, Average, Difficult.
6. Questions can optionally include images.
7. Each question has exactly 4 multiple-choice options with one correct answer.
8. Questions are immediately available for games after creation.

Preconditions

1. Admin is logged into the admin panel (has active admin session).
2. Question bank database table exists and is accessible.
3. Categories and difficulty levels are pre-configured in the system.
4. Admin is on the Questions management page.

Post-conditions

1. A new question is created and stored in the question bank database.
2. Question is added to the available questions list.
3. Question can be used based on its category and difficulty settings.
4. System displays confirmation message to admin.
5. Questions list is updated to show the new question.
6. System is ready to accept additional admin actions.

Scenario 1: Admin Successfully Adds New Question to Question Bank

Base Flow

Step	Executor	Action/Description	Data (Input or Output)
1	Admin	Navigates to admin panel and clicks on "Questions" menu item.	Input: Click action on Questions
2	System	Retrieves and displays list of all existing questions with filters.	Output: Questions list table
3	Admin	Clicks "Add New Question" button.	Input: Click action on Add New Question
4	System	Displays question entry form with all required and optional fields.	Output: Question form (question text area, option fields, category dropdown (Science/Math), year level dropdown, difficulty dropdown (Easy/Average/Difficult), image upload field (optional), "Save Question" button, "Cancel" button)
5	Admin	Enters question text, fills in all 4 answer options, selects the correct answer, chooses category, year level, and difficulty, optionally uploads an image, then clicks "Save Question" button.	Input: question text, option text, correct answer selection, category selection, year level selection, difficulty selection, image file (optional)
6	System	Validates all input fields: checks that question text is not empty, all 4 options are filled, correct answer is selected, category is selected, year level is selected, difficulty is selected, image format is valid (if uploaded).	Input: All form data from Step 5
7	System	Validates that question text is unique or not a duplicate of existing questions.	Input: Question text; Output: Uniqueness check (PASS - no exact duplicate found)
8	System	If image was uploaded: processes image file (validates file type jpg/png, resizes if needed, stores in image storage).	Input: Image file; Output: Image URL or stored image path
9	System	Creates new question record in question bank database.	Input: Question text, options, correct answer, category, year level, difficulty, image URL (if any), creator Admin ID; Output: New question record
10	System	Displays success message: "Question has been added successfully to the question bank!"	Output: Success message with new Question ID
11	System	Redirects admin back to Questions list page showing	Output: Updated questions list table with new question entry

		updated list with new question included.	
--	--	--	--

Alternative Flows

Base Step	Alt Flow Step	Executor	Action/Description	Data (Input and/or Output)
6			Condition: Required fields are empty or invalid	
	1	System	Identifies missing required fields or invalid data.	Input: Incomplete or invalid form data
	2	System	Displays error message: "Please fill in all required fields" with field-specific errors (e.g., "Question text is required", "All 4 options must be filled", "Please select correct answer") and highlights problematic fields in red.	Output: Error message, field highlighting
	3	System	Returns to Step 5 of base flow.	Output: Question form with existing data preserved

UC-MQS2: Edit Question Info**Use Case Name**

Edit Question Info

Use Case ID

UC-MQS2

Description**Goal**

To allow a logged-in admin to update the details of an existing quiz question in the STARBOOKS system, ensuring question content remains accurate, relevant, and properly categorized.

Actions/Short Summary

- Admin navigates to question management section
- Admin selects specific question to edit
- System displays current question information in editable form
- Admin modifies desired fields

- System validates the updated information
- System saves changes to database
- System confirms successful question update

Trigger

Admin clicks the "Edit" icon next to a question entry in the Questions list because they need to correct errors, update content, recategorize the question, or improve answer options.

Actor(s)

1. **Primary Actor** - Admin
2. **Secondary Actors** - System

Assumptions

1. The system is available and admin session is active.
2. Question bank contains at least one existing question.
3. All question fields can be edited.
4. Updated questions take effect immediately in games.

Preconditions

1. Admin is logged into the admin panel (has active admin session).
2. Target question exists in the question bank database.
3. Admin is on the Questions management page.
4. System can retrieve current question data.

Post-conditions

1. Selected question information is updated in the database.
2. System displays confirmation message to admin.
3. Updated question information is reflected in the Questions list.
4. Updated question is available for future game with new content.
5. System is ready to accept additional admin actions.

Scenario 1: Admin Successfully Updates Existing Question**Base Flow**

Step	Executor	Action/Description	Data (Input or Output)
1	Admin	Navigates to "Questions" from the admin panel menu.	Input: Click action on Questions
2	System	Retrieves and displays list of all existing questions with filters.	Output: Questions list table

3	Admin	Locates the target question to edit and clicks the "Edit" icon for that row.	Input: Click action on Edit icon for selected question
4	System	Retrieves selected question's current data from database.	Input: Question ID; Output: Complete question data (question text, options, correct answer, category, year level, difficulty, image URL if exists)
5	System	Displays Edit Question form with fields pre-populated with current information.	Output: Edit question form (question text area with current text, option fields with current options, correct answer dropdown pre-selected, category dropdown pre-selected, year level dropdown pre-selected, difficulty dropdown pre-selected, current image displayed with "Change Image" option, "Save Changes" button, "Cancel" button)
6	Admin	Modifies one or more fields (question text, options, correct answer, category, year level, difficulty, image) and clicks "Save Changes" button.	Input: Updated values for modified fields
7	System	Validates all input fields: checks that question text is not empty, all 4 options are filled, correct answer is selected, category is selected, year level is selected, difficulty is selected, image format is valid (if changed).	Input: Updated question data from Step 6
8	System	Validates that all 4 answer options have unique text (no duplicates).	Input: Options; Output: Uniqueness validation (PASS)
9	System	If image was changed: processes new image file (validates file type, resizes if needed, stores in image storage, deletes old image if applicable).	Input: New image file (if provided); Output: New image URL or stored image path
10	System	Compares new data with existing data and identifies changed fields.	Input: New question data, existing question data; Output: List of changed fields
11	System	Updates question record in database with new information.	Input: Question ID, updated field values; Output: Updated question record

12	System	Displays success message: "Question has been updated successfully!"	Output: Success message with Question ID
13	System	Redirects admin back to Questions list page showing updated information.	Output: Updated questions list table with modified question data displayed

Alternative Flows

Base Step	Alt Flow Step	Executor	Action/Description	Data (Input and/or Output)
7			Condition: Invalid or empty required fields	
	1	System	Identifies required fields that are empty or have invalid values.	Input: Question form data from Step 6
	2	System	Displays validation error message: "Please correct the following errors:" followed by field-specific messages and highlights invalid fields in red.	Output: Error message, field highlighting
	3	System	Returns to Step 6 of base flow.	Output: Edit question form with entered data preserved

UC-MQS3: Delete Question**Use Case Name**

Delete Question

Use Case ID

UC-MQS3

Description**Goal**

To allow a logged-in admin to remove a quiz question from the question bank in the STARBOOKS system, maintaining question quality by removing outdated, incorrect, or inappropriate content.

Actions/Short Summary

- Admin navigates to question management section
- Admin selects specific question to delete

- System displays confirmation dialog with warning
- Admin confirms deletion intention
- System validates deletion is allowed
- System removes question from database
- System updates questions list display

Trigger

Admin clicks the "Delete" icon next to a question entry in the Questions list because the question is outdated, incorrect, duplicate, inappropriate, or no longer needed for the quiz content.

Actor(s)

1. **Primary Actor** - Admin
2. **Secondary Actors** - System

Assumptions

1. The system is available and admin session is active.
2. Question bank contains at least one existing question.
3. Questions can be deleted even if previously used in games.
4. Deletion is permanent.
5. Deleting a question does not affect historical player statistics from past games.

Preconditions

1. Admin is logged into the admin panel (has active admin session).
2. Target question exists in the question bank database.
3. Admin is on the Questions management page.
4. Question bank contains at least one question.

Post-conditions

1. Selected question is removed from the question bank database.
2. Question is no longer available for use in future games.
3. Questions list is updated to reflect the deletion.
4. System is ready to accept additional admin actions.

Scenario 1: Admin Successfully Deletes Question**Base Flow**

Step	Executor	Action/Description	Data (Input or Output)
1	Admin	Navigates to "Questions" from the admin panel menu.	Input: Click action on Questions

2	System	Retrieves and displays list of all existing questions with filters.	Output: Questions list table
3	Admin	Locates the target question to delete and clicks the "Delete" icon for that row.	Input: Click action on Delete icon for selected question
4	System	Retrieves question details for display in confirmation dialog.	Input: Question ID; Output: Question data (question text, category, difficulty, year level)
5	System	Displays deletion confirmation dialog with warning message and question preview.	Output: Confirmation dialog ("Are you sure you want to delete this question? This action cannot be undone.", question text preview, category/difficulty displayed, "Confirm Delete" button, "Cancel" button)
6	Admin	Reviews the confirmation dialog and question details, then clicks "Confirm Delete" button.	Input: Click action on Confirm Delete button
7	System	Retrieves complete question data including image URL (if exists) for deletion processing.	Input: Question ID; Output: Complete question record (all fields, image URL)
8	System	If question has associated image: deletes image file from storage.	Input: Image URL; Output: Image file deleted from storage
9	System	Deletes the question record from database.	Input: Question ID, deleting Admin ID; Output: Question record removed
10	System	Displays success message: "Question has been deleted successfully."	Output: Success message
11	System	Refreshes Questions list page showing updated list without the deleted question.	Output: Updated questions list table (deleted question removed from display)

Alternative Flows

Base Step	Alt Flow Step	Executor	Action/Description	Data (Input and/or Output)
6			Condition: Admin cancels deletion	

	1	Admin	Reviews confirmation dialog and clicks "Cancel" button instead of "Confirm Delete".	Input: Click action on Cancel button
	2	System	Closes the confirmation dialog without making any changes.	Input: None
	3	System	Returns admin to Questions list page with all questions unchanged.	Output: Questions list table unchanged
	4	Use case ends (no deletion performed).		

UC-MQS4: Set Number of Questions

Use Case Name

Set Number of Questions

Use Case ID

UC-MQS4

Description

Goal

To allow a logged-in admin to configure the number of questions that will be presented to players in quiz games for each difficulty level.

Actions/Short Summary

- Admin navigates to difficulty settings
- System displays current question count settings per difficulty level
- Admin modifies number of questions for one or more difficulty levels
- System validates that sufficient questions exist in question bank
- System saves new configuration settings
- System confirms successful update

Trigger

Admin navigates to "Difficulty Settings" because they need to adjust game parameters, balance game difficulty, or update the number of questions based on available content in the question bank.

Actor(s)

1. **Primary Actor** - Admin
2. **Secondary Actors** - System

Assumptions

1. The system is available and admin session is active.
2. Difficulty levels (Easy, Average, Difficult) are pre-configured in the system.
3. Number of questions applies to quiz-based games (Whiz Challenge, Whiz Battle).
4. System validates against actual available questions in question bank.
5. Reasonable limits exist.

Preconditions

1. Admin is logged into the admin panel (has active admin session).
2. Difficulty levels exist in the database.
3. Question bank contains at least some questions.
4. Admin is on the Difficulty Settings or Quiz Configuration page.

Post-conditions

1. Number of questions per difficulty level is updated in configuration database.
2. System displays confirmation message to admin.
3. New settings are applied to future games immediately.
4. System is ready to accept additional configuration changes.

Scenario 1: Admin Successfully Updates Number of Questions for Difficulty Levels**Base Flow**

Step	Executor	Action/Description	Data (Input or Output)
1	Admin	Navigates to admin panel and clicks on "Difficulty Settings" or "Quiz Configuration" menu item.	Input: Click action on Difficulty Settings
2	System	Retrieves current configuration settings from database.	Input: None; Output: Current settings (Easy questions count, Average questions count, Difficult questions count)
3	System	Counts available questions in question bank for each difficulty level.	Input: Question bank query by difficulty; Output: Available question counts (Easy: X questions, Average: Y questions, Difficult: Z questions)
4	System	Displays difficulty settings page with current values and available question counts.	Output: Settings form (Easy difficulty: input field with current value, available questions count displayed, Average difficulty: input field with current value, available questions count displayed, Difficult difficulty: input field with current value, available questions count displayed)

			count displayed, "Save Settings" button, "Reset to Default" button)
5	Admin	Modifies the number of questions for one or more difficulty levels by entering new values, then clicks "Save Settings" button.	Input: New question counts (Easy: new value, Average: new value, Difficult: new value)
6	System	Validates all input values: checks that values are numeric, positive integers, within allowed range (e.g., 1-50).	Input: New question counts from Step 5; Output: Format validation (PASS)
7	System	For each difficulty level, validates that requested number does not exceed available questions in question bank.	Input: Requested question counts, available question counts; Output: Availability validation (PASS for all levels)
8	System	Compares new settings with current settings to identify changes.	Input: New settings, current settings; Output: List of changed difficulty levels
9	System	Updates configuration settings in database.	Input: New question counts for each difficulty level; Output: Updated configuration record with modification timestamp, modifier Admin ID
10	System	Displays success message: "Number of questions per difficulty level has been updated successfully!"	Output: Success message showing new values (Easy: X, Average: Y, Difficult: Z)
11	System	Refreshes the settings page showing updated configuration values.	Output: Updated settings form with new values displayed

Alternative Flows

Base Step	Alt Flow Step	Executor	Action/Description	Data (Input and/or Output)
6			Condition: Invalid input format (non-numeric or negative value)	
	1	System	Validates input and detects non-numeric characters or negative values.	Input: Question count input; Output: Format validation error
	2	System	Displays error message: "Please enter a valid positive number for number of	Output: Error message, field highlighting

			questions." and highlights invalid field in red.	
	3	System	Returns to Step 5 of base flow.	Output: Settings form with entered data preserved
6			Condition: Value exceeds allowed range	
	1	System	Validates input and detects value is outside allowed range (e.g., 0 or > 50).	Input: Question count input; Output: Range validation error
	2	System	Displays error message: "Number of questions must be between 1 and 50."	Output: Error message
	3	System	Returns to Step 5 of base flow.	Output: Settings form
7			Condition: Requested number exceeds available questions	
	1	System	Validates requested number against available questions and finds insufficient questions for one or more difficulty levels.	Input: Requested count, available count; Output: Insufficient questions detected (e.g., requested 20 for Easy but only 15 available)
	2	System	Displays warning message: "Not enough questions available. [Difficulty] level: Requested [X] but only [Y] questions available. Please reduce the number or add more questions."	Output: Warning message with specific details for each problematic difficulty level
	3	System	Does not save changes for affected difficulty levels.	Input: None
	4	System	Returns to Step 5 of base flow.	Output: Settings form

UC-MQS5: Set Stars**Use Case Name**

Set Stars

Use Case ID

UC-MQS5

Description**Goal**

To allow a logged-in admin to configure the star/point value awarded to players for correct answers at each difficulty level, ensuring appropriate reward progression and game balance based on question difficulty.

Actions/Short Summary

- Admin navigates to difficulty settings
- System displays current star values per difficulty level
- Admin modifies star values for one or more difficulty levels
- System validates input values
- System saves new star configuration
- System confirms successful update

Trigger

Admin navigates to "Difficulty Settings" or quiz configuration page because they need to adjust reward values, balance game scoring, or modify incentive structure based on difficulty levels.

Actor(s)

1. **Primary Actor** - Admin
2. **Secondary Actors** - System

Assumptions

1. The system is available and admin session is active.
2. Difficulty levels (Easy, Average, Difficult) are pre-configured in the system.
3. Stars represent points or score values awarded per correct answer.
4. Star values apply to Whiz Battle.
5. Changes take effect immediately.
6. Reasonable limits exist.

Preconditions

1. Admin is logged into the admin panel (has active admin session).
2. Difficulty levels exist in the database.
3. Admin is on the Difficulty Settings.

Post-conditions

1. Star values per difficulty level are updated in configuration database.
2. System displays confirmation message to admin.
3. New star values are applied immediately.
4. System is ready to accept additional configuration changes.

Scenario 1: Admin Successfully Updates Star Values for Difficulty Levels**Base Flow**

Step	Executor	Action/Description	Data (Input or Output)
1	Admin	Navigates to admin panel and clicks on "Difficulty Settings" menu item.	Input: Click action on Difficulty Settings
2	System	Retrieves current star value configuration from database.	Input: None; Output: Current star values (Easy: X stars, Average: Y stars, Difficult: Z stars)
3	System	Displays difficulty settings page with current star values.	Output: Settings form (Easy difficulty: input field with current star value, Average difficulty: input field with current star value, Difficult difficulty: input field with current star value, "Save Settings" button, "Reset to Default" button)
4	Admin	Modifies the star values for one or more difficulty levels by entering new values, then clicks "Save Settings" button.	Input: New star values (Easy: new value, Average: new value, Difficult: new value)
5	System	Validates all input values: checks that values are numeric, positive integers (greater than 0), within allowed range (e.g., 1-100).	Input: New star values from Step 4; Output: Validation result (PASS)
6	System	Compares new settings with current settings to identify changes.	Input: New star values, current star values; Output: List of changed difficulty levels
7	System	Updates star value configuration in database.	Input: New star values for each difficulty level; Output: Updated configuration record
8	System	Displays success message: "Star values per difficulty level have been updated successfully!"	Output: Success message showing new values (Easy: X stars, Average: Y stars, Difficult: Z stars)
9	System	Refreshes the settings page showing updated star value configuration.	Output: Updated settings form with new values displayed

Alternative Flows

Base Step	Alt Flow Step	Executor	Action/Description	Data (Input and/or Output)
-----------	---------------	----------	--------------------	----------------------------

5			Condition: Invalid input format (non-numeric value)	
	1	System	Validates input and detects non-numeric characters.	Input: Star value input; Output: Format validation error
	2	System	Displays error message: "Please enter a valid positive number for star value." and highlights invalid field in red.	Output: Error message, field highlighting
	3	System	Returns to Step 4 of base flow.	Output: Settings form with entered data preserved
5			Condition: Negative or zero star value entered	
	1	System	Validates input and detects value is zero or negative.	Input: Star value input; Output: Validation error (≤ 0)
	2	System	Displays error message: "Star value must be greater than 0. Please enter a positive number." and highlights invalid field in red.	Output: Error message, field highlighting
	3	System	Returns to Step 4 of base flow.	Output: Settings form
5			Condition: Star value exceeds allowed maximum	
	1	System	Validates input and detects value exceeds maximum limit (e.g., > 100).	Input: Star value input; Output: Range validation error
	2	System	Displays error message: "Star value must be between 1 and 100."	Output: Error message
	3	System	Returns to Step 4 of base flow.	Output: Settings form

UC-MQS6: Set Timer**Use Case Name**

Set Timer

Use Case ID

UC-MQS6

Description

Fully Dressed Use Cases for STARBOOKS Whiz Challenge
Last Date Updated:6

Page 72 of 102
Template Version: 1.0

Goal

To allow a logged-in admin to configure the time limit for each difficulty level in quiz games, ensuring appropriate challenge and time pressure based on question difficulty.

Actions/Short Summary

- Admin navigates to difficulty settings
- System displays current timer values per difficulty level
- Admin modifies time limits for one or more difficulty levels
- System validates input values
- System saves new timer configuration
- System confirms successful update

Trigger

Admin navigates to "Difficulty Settings" page because they need to adjust time constraints, and balance game difficulty through time pressure.

Actor(s)

1. **Primary Actor** - Admin
2. **Secondary Actors** - System

Assumptions

1. The system is available and admin session is active.
2. Difficulty levels (Easy, Average, Difficult) are pre-configured in the system.
3. Timer represents seconds allowed per question.
4. Timer applies to timed quiz games (Whiz Challenge, Whiz Battle).
5. Changes take effect immediately.
6. Reasonable limits exist.
7. Timer specification is in seconds.

Preconditions

1. Admin is logged into the admin panel (has active admin session).
2. Difficulty levels exist in the database.
3. Admin is on the Difficulty Settings.

Post-conditions

1. Timer values per difficulty level are updated in configuration database.
2. System displays confirmation message to admin.
3. New timer values are applied immediately.

4. System is ready to accept additional configuration changes.

Scenario 1: Admin Successfully Updates Timer Values for Difficulty Levels

Base Flow

Step	Executor	Action/Description	Data (Input or Output)
1	Admin	Navigates to admin panel and clicks on "Difficulty Settings" menu item.	Input: Click action on Difficulty Settings
2	System	Retrieves current timer configuration from database.	Input: None; Output: Current timer values (Easy: X seconds, Average: Y seconds, Difficult: Z seconds)
3	System	Displays difficulty settings page with current timer values.	Output: Settings form (Easy difficulty: input field with current timer value in seconds, Average difficulty: input field with current timer value in seconds, Difficult difficulty: input field with current timer value in seconds, "Save Settings" button, "Reset to Default" button)
4	Admin	Modifies the timer values for one or more difficulty levels by entering new values in seconds, then clicks "Save Settings" button.	Input: New timer values in seconds (Easy: new value, Average: new value, Difficult: new value)
5	System	Validates all input values: checks that values are numeric, positive integers (greater than 0), within allowed range (e.g., 5-300 seconds).	Input: New timer values from Step 4; Output: Validation result (PASS)
6	System	Compares new settings with current settings to identify changes.	Input: New timer values, current timer values; Output: List of changed difficulty levels
7	System	Updates timer configuration in database.	Input: New timer values for each difficulty level; Output: Updated configuration record with modification timestamp, modifier Admin ID
8	System	Displays success message: "Timer settings per difficulty level have been updated successfully!"	Output: Success message showing new values (Easy: X seconds, Average: Y seconds, Difficult: Z seconds)
9	System	Refreshes the settings page showing updated timer configuration.	Output: Updated settings form with new values displayed

Alternative Flows

Base Step	Alt Flow Step	Executor	Action/Description	Data (Input and/or Output)
5			Condition: Invalid input format (non-numeric value)	
	1	System	Validates input and detects non-numeric characters.	Input: Timer value input; Output: Format validation error
	2	System	Displays error message: "Please enter a valid number for timer value (in seconds)." and highlights invalid field in red.	Output: Error message, field highlighting
	3	System	Returns to Step 4 of base flow.	Output: Settings form with entered data preserved
5			Condition: Negative or zero timer value entered	
	1	System	Validates input and detects value is zero or negative.	Input: Timer value input; Output: Validation error (≤ 0)
	2	System	Displays error message: "Timer value must be greater than 0 seconds. Please enter a positive number." and highlights invalid field in red.	Output: Error message, field highlighting
	3	System	Returns to Step 4 of base flow.	Output: Settings form
5			Condition: Timer value below minimum threshold	
	1	System	Validates input and detects value is below minimum (e.g., < 5 seconds).	Input: Timer value input; Output: Minimum threshold error
	2	System	Displays error message: "Timer value must be at least 5 seconds to allow reasonable time to answer."	Output: Error message
	3	System	Returns to Step 4 of base flow (form remains populated with entered data).	Output: Settings form
5			Condition: Timer value excessively high	

	1	System	Validates input and detects value exceeds maximum limit.	Input: Timer value input; Range validation error
	2	System	Displays error message: "Timer value must not exceed 300 seconds (5 minutes). Please enter a lower value."	Output: Error message
	3	System	Returns to Step 4 of base flow.	Output: Settings form

UC-MPL1: Add New Player

Use Case Name

Add New Player

Use Case ID

UC-MPL1

Description

Goal

To allow a logged-in admin to create a new player account in the STARBOOKS system, enabling the new player to access and play games without going through the self-registration process.

Actions/Short Summary

- Admin navigates to player management section
- Admin initiates new player creation
- System displays player registration form
- Admin enters new player details
- System validates the input data
- System creates new player account
- System confirms successful account creation

Trigger

Admin clicks "Add New Player" button from the Player List page because they need to manually create player accounts for users.

Actor(s)

1. **Primary Actor** - Admin
2. **Secondary Actors** - System

Assumptions

1. The system is available and admin session is active.
2. Player registration fields match those in UC-MPA1 Player Register.
3. Passwords must meet the same security requirements.
4. Username must be unique across all player accounts.
5. Newly created accounts are active immediately.
6. Players can login using the credentials provided by the admin.

Preconditions

1. Admin is logged into the admin panel (has active admin session).
2. Player list database table exists and is accessible.
3. Admin is on the Player List page.

Post-conditions

1. A new player account is created and stored in the player database.
2. Player account is added to the Player List.
3. Player can login using the created credentials.
4. System displays confirmation message to admin.
5. Player List is updated to show the new player.
6. System is ready to accept additional admin actions.

Scenario 1: Admin Successfully Creates New Player Account**Base Flow**

Step	Executor	Action/Description	Data (Input or Output)
1	Admin	Navigates to admin panel and clicks on "Player List" menu item.	Input: Click action on Player List
2	System	Retrieves and displays list of all existing player accounts.	Output: Player list table
3	Admin	Clicks "Add New Player" button.	Input: Click action on Add New Player
4	System	Displays player registration form with all required fields.	Output: Player registration form (username field, password field, confirm password field, school name field, age field, category dropdown, sex dropdown, region dropdown, province dropdown, city dropdown, avatar selection, "Create Player" button, "Cancel" button)

5	Admin	Enters all required details for the new player: username, password, confirms password, school name, age, category, sex, region, province, city, selects avatar, then clicks "Create Player" button.	Input: username, password, confirm password, school name, age, category, sex, region, province, city, avatar selection
6	System	Validates all input fields: checks that all required fields are filled, username format is valid, username is unique, password meets security requirements, passwords match, age is within valid range, all selections are valid options.	Input: All form data from Step 5
7	System	Checks username uniqueness by querying player database to ensure username does not already exist.	Input: Username; Output: Username availability check (AVAILABLE)
8	System	Hashes the player password using secure hashing algorithm.	Input: Password (plain text); Output: Hashed password
9	System	Creates new player account record in player database.	Input: Username, hashed password, school name, age, category, sex, region, province, city, avatar, creator Admin ID; Output: New player record
10	System	Initializes player statistics and profile	Input: Player ID; Output: Initialized player stats record
11	System	Displays success message: "New player account '[Username]' has been created successfully!"	Output: Success message with new player username
12	System	Redirects admin back to Player List page showing updated list with new player included.	Output: Updated player list table with new player entry

Alternative Flows

Base Step	Alt Flow Step	Executor	Action/Description	Data (Input and/or Output)
6			Condition: Required fields are empty or invalid	
	1	System	Identifies missing required fields or invalid formats.	Input: Incomplete or invalid form data

	2	System	Displays error message: "Please fill in all required fields" or field-specific error messages (e.g., "Invalid age format") and highlights problematic fields in red.	Output: Error message, highlighting field
	3	System	Returns to Step 5 of base flow.	Output: Player registration form with existing data preserved
7			Condition: Username already exists	
	1	System	Queries player database and finds username already in use.	Input: Username; Output: Username already exists
	2	System	Displays error message: "Username '[Username]' already exists. Please choose a different username."	Output: Error message with username highlighted
	3	System	Returns to Step 5 of base flow.	Output: Player registration form with data preserved
6			Condition: Password does not meet security requirements	
	1	System	Validates password against security rules (minimum 8 characters, complexity requirements).	Input: Password; Output: Validation result (FAIL)
	2	System	Displays error message: "Password must be at least 8 characters and include uppercase, lowercase, and numbers."	Output: Error message with password requirements
	3	System	Returns to Step 5 of base flow.	Output: Player registration form with data preserved
6			Condition: Passwords do not match	
	1	System	Compares password field with confirm password field and finds mismatch.	Input: Password, confirm password; Output: Mismatch detected
	2	System	Displays error message: "Passwords do not match. Please re-enter."	Output: Error message
	3	System	Clears password and confirm password fields.	Output: Player registration form with passwords cleared, other data preserved

	4	System	Returns to Step 5 of base flow.	Output: Player registration form
--	---	--------	---------------------------------	----------------------------------

UC-MPL2: Edit Player Profile

Use Case Name

Edit Player Profile

Use Case ID

UC-MPL2

Description

Goal

To allow a logged-in admin to update an existing player's profile information in the STARBOOKS system, ensuring player account details remain accurate, current, and properly configured.

Actions/Short Summary

- Admin navigates to player management section
- Admin selects specific player profile to edit
- System displays current player information in editable form
- Admin modifies desired fields
- System validates the updated information
- System saves changes to database
- System confirms successful profile update

Trigger

Admin clicks the "Edit" icon/button next to a player entry in the Player List because they need to correct errors, update player information, or assist players with profile changes.

Actor(s)

1. **Primary Actor** - Admin
2. **Secondary Actors** - System

Assumptions

1. The system is available and admin session is active.
2. Player list contains at least one existing player account.
3. All profile fields have defined validation rules.
4. Admins can edit any player profile.

Preconditions

1. Admin is logged into the admin panel (has active admin session).
2. Target player account exists in the database.
3. Admin is on the Player List page.
4. System can retrieve current player profile data.

Post-conditions

1. Selected player profile information is updated in the database.
2. System displays confirmation message to admin.
3. Updated profile information is reflected in the Player List.
4. Player can see updated information when they login.
5. System is ready to accept additional admin actions.

Scenario 1: Admin Successfully Updates Player Profile

Base Flow

Step	Executor	Action/Description	Data (Input or Output)
1	Admin	Navigates to "Player List" from the admin panel menu.	Input: Click action on Player List
2	System	Retrieves and displays list of all existing player accounts.	Output: Player list table
3	Admin	Locates the target player profile to edit and clicks the "Edit" icon for that row.	Input: Click action on Edit icon for selected player
4	System	Retrieves selected player's current profile data from database.	Input: Player ID; Output: Complete player profile data (username, school name, age, category, sex, region, province, city, avatar, registration date, status)
5	System	Displays Edit Player Profile form with fields pre-populated with current information.	Output: Edit player form (username field, school name field with current value, age field with current value, category dropdown pre-selected, sex dropdown pre-selected, region dropdown pre-selected, province dropdown pre-selected, city dropdown pre-selected, avatar selection showing current avatar, "Save Changes" button, "Cancel" button)
6	Admin	Modifies one or more editable fields (username, school name, age, category, sex, region, province, city, avatar) and clicks "Save Changes" button.	Input: Updated values for modified fields

7	System	Validates all input fields: checks that required fields are not empty, age is within valid range, selections are valid options, data formats are correct.	Input: Updated profile data from Step 6
8	System	Compares new data with existing data and identifies changed fields.	Input: New profile data, existing profile data; Output: List of changed fields
9	System	Updates player profile record in database with new information.	Input: Player ID, updated field values; Output: Updated player profile record
10	System	Displays success message: "Player profile '[Username]' has been updated successfully!"	Output: Success message with player username
11	System	Redirects admin back to Player List page showing updated information.	Output: Updated player list table with modified profile data displayed

Alternative Flows

Base Step	Alt Flow Step	Executor	Action/Description	Data (Input and/or Output)
7			Condition: Invalid or empty required fields	
	1	System	Identifies required fields that are empty or have invalid values.	Input: Profile form data from Step 6
	2	System	Displays validation error message: "Please correct the following errors:" followed by field-specific messages and highlights invalid fields in red.	Output: Error message, field highlighting
	3	System	Returns to Step 6 of base flow.	Output: Edit player form with entered data preserved
7			Condition: Invalid input format (age, selections, etc.)	
	1	System	Validates input formats and detects invalid data (age not numeric, invalid selections, etc.).	Input: Profile form data
	2	System	Displays error message specifying which field(s) have invalid format.	Output: Error message with field-specific details

	3	System	Returns to Step 6 of base flow.	Output: Edit player form
--	---	--------	---------------------------------	--------------------------

UC-MPL3: Change Player Password

Use Case Name

Change Player Password

Use Case ID

UC-MPL3

Description

Goal

To allow a logged-in admin to change a player's account password in the STARBOOKS s system, typically when a player has forgotten their password or requests a password reset, ensuring account security while providing account recovery assistance.

Actions/Short Summary

- Admin navigates to player management section
- Admin selects specific player account for password change
- System displays password change form
- Admin enters current password and new password (with player assistance)
- System verifies current password is correct
- System validates new password meets security requirements
- System updates password in database
- System confirms successful password change

Trigger

Admin clicks the "Password" icon/button next to a player entry in the Player List because the player has forgotten their password, requested a password reset, or needs password assistance from admin.

Actor(s)

1. **Primary Actor** - Admin
2. **Secondary Actors**
 - System
 - Player (provides current password verification and new password preference)

Assumptions

1. The system is available and admin session is active.

2. Player is present with admin or communicates with admin to provide current password.
3. Passwords are stored securely using encryption/hashing in the database.
4. The system has defined password security requirements.
5. Admin requires current password to change player password.
6. Password change takes effect immediately.

Preconditions

1. Admin is logged into the admin panel (has active admin session).
2. Target player account exists in the database with an existing password.
3. Admin is on the Player List page.
4. Player is available to provide current password verification.

Post-conditions

1. Player's password is updated in the database with new hashed password.
2. Old password is replaced and can no longer be used for login.
3. System displays confirmation message to admin.
4. Player can login with new password immediately.
5. System is ready to accept additional admin actions.

Scenario 1: Admin Successfully Changes Player Password with Player Assistance

Base Flow

Step	Executor	Action/Description	Data (Input or Output)
1	Admin	Navigates to "Player List" from the admin panel menu.	Input: Click action on Player List
2	System	Retrieves and displays list of all existing player accounts.	Output: Player list table
3	Admin	Locates the target player account and clicks the "Password" icon for that row.	Input: Click action on Password icon for selected player
4	System	Displays Change Password dialog/form with input fields.	Output: Change password form (player username displayed for reference, current password field, new password field, confirm new password field, "Update Password" button, "Cancel" button)
5	Admin	With player's assistance, enters player's current password, new password, and confirms new	Input: current password (provided by player), new password (chosen by player or admin), confirm new password

		password by re-entering it. Clicks "Update Password" button.	
6	System	Validates that all three fields are not empty.	Input: Form data from Step 5
7	System	Retrieves player's stored hashed password from database.	Input: Player ID; Output: Stored hashed password
8	System	Verifies entered current password matches the stored hashed password.	Input: Entered current password, stored hashed password
9	System	Validates new password matches confirm new password field (ensures no typos).	Input: New password, confirm new password
10	System	Validates new password meets security requirements (minimum 8 characters, includes uppercase, lowercase, and numbers).	Input: New password; Output: Validation result (PASS)
11	System	Checks that new password is different from current password.	Input: New password, current password; Output: Different passwords confirmed
12	System	Hashes the new password using secure hashing algorithm.	Input: New password (plain text); Output: Hashed new password
13	System	Updates player's password in database with the new hashed password.	Input: Player ID, new hashed password; Output: Updated password record with modification timestamp, modifier Admin ID
14	System	Displays success message: "Password for player '[Username]' has been changed successfully!"	Output: Success message
15	System	Closes the Change Password dialog and returns admin to Player List page.	Output: Player list table

Alternative Flows

Base Step	Alt Flow Step	Executor	Action/Description	Data (Input and/or Output)
6			Condition: Empty password fields	
	1	System	Identifies one or more password fields are empty.	Input: Form data from Step 5
	2	System	Displays error message: "Please fill in all password fields."	Output: Error message

	3	System	Returns to Step 5 of base flow.	Output: Change password form
8			Condition: Incorrect current password	
	1	System	Compares entered current password with stored hashed password and finds no match.	Input: Entered current password, stored hashed password
	2	System	Displays error message: "Current password is incorrect. Please verify with the player and try again."	Output: Error message
	3	System	Clears all password fields for security.	Output: Change password form with empty fields
	4	System	Returns to Step 5 of base flow.	Output: Change password form
9			Condition: New password and confirm password do not match	
	1	System	Compares new password field with confirm new password field and finds mismatch.	Input: New password, confirm new password
	2	System	Displays error message: "New passwords do not match. Please re-enter."	Output: Error message
	3	System	Clears new password and confirm new password fields.	Output: Change password form with current password preserved
	4	System	Returns to Step 5 of base flow.	Output: Change password form
10			Condition: New password does not meet security requirements	
	1	System	Validates new password against security rules and identifies violations.	Input: New password
	2	System	Displays error message: "Password must be at least 8 characters and include uppercase, lowercase, and numbers."	Output: Error message with specific requirements
	3	System	Clears new password and confirm new password fields.	Output: Change password form with current password preserved

	4	System	Returns to Step 5 of base flow.	Output: Change password form
11			Condition: New password is same as current password	
	1	System	Compares new password with current password and finds they are identical.	Input: New password, current password
	2	System	Displays error message: "New password must be different from current password."	Output: Error message
	3	System	Clears new password and confirm new password fields.	Output: Change password form with current password preserved
	4	System	Returns to Step 5 of base flow.	Output: Change password form

UC-MPL4: Delete Player

Use Case Name

Delete Player

Use Case ID

UC-MPL4

Description

Goal

To allow a logged-in admin to remove a player account from the STARBOOKS system, typically when requested by the player, for account cleanup, or data management purposes, while maintaining data integrity and audit trails.

Actions/Short Summary

- Admin navigates to player management section
- Admin selects specific player account to delete
- System displays confirmation dialog with warning
- Admin confirms deletion intention
- System validates deletion is allowed
- System removes player account
- System updates player list display

Trigger

Admin clicks the "Delete" icon next to a player entry in the Player List because the player has requested account deletion, the account is inactive/duplicate, or for data management purposes.

Actor(s)

1. **Primary Actor** - Admin
2. **Secondary Actors** - System

Assumptions

1. The system is available and admin session is active.
2. Player list contains at least one existing player account.
3. Player data is important - system should confirm before deletion.
4. Deleted players cannot login.

Preconditions

1. Admin is logged into the admin panel (has active admin session).
2. Target player account exists in the database.
3. Admin is on the Player List page.

Post-conditions

1. Selected player account is removed in the database.
2. Player can no longer login to the system.
3. Player is removed from the Player List display.
4. Historical game data and statistics may be preserved for reporting.
5. System is ready to accept additional admin actions.

Scenario 1: Admin Successfully Deletes Player Account**Base Flow**

Step	Executor	Action/Description	Data (Input or Output)
1	Admin	Navigates to "Player List" from the admin panel menu.	Input: Click action on Player List
2	System	Retrieves and displays list of all existing player accounts.	Output: Player list table
3	Admin	Locates the target player account to delete and clicks the "Delete" icon for that row.	Input: Click action on Delete icon for selected player
4	System	Retrieves player details for display in confirmation dialog.	Input: Player ID; Output: Player data (username, school name, registration date, games played, badges earned)

5	System	Displays deletion confirmation dialog with warning message and player preview.	Output: Confirmation dialog ("Are you sure you want to delete player '[Username]'? This action cannot be undone.", player details displayed - username, school, games played, badges earned, "Confirm Delete" button, "Cancel" button, optional "Reason for deletion" text field)
6	Admin	Reviews the confirmation dialog and player details, optionally enters deletion reason, then clicks "Confirm Delete" button.	Input: Click action on Confirm Delete button, optional deletion reason text
7	System	Retrieves complete player data including all associated records.	Input: Player ID; Output: Complete player record (profile, statistics, game history, badges, rewards)
9	System	Hard-deletes the player account based on system configuration.	Input: Player ID; Output: Player account removed
10	System	Invalidates all active sessions for the deleted player (if currently logged in).	Input: Player ID; Output: Sessions terminated, session tokens invalidated
11	System	Updates any related records (e.g., mark rewards as "Player Deleted", preserve game history with anonymized reference).	Input: Player ID; Output: Related records updated
12	System	Displays success message: "Player account '[Username]' has been deleted successfully."	Output: Success message with deleted player username
13	System	Refreshes Player List page showing updated list without the deleted player.	Output: Updated player list table (deleted player removed from display)

Alternative Flows

Base Step	Alt Flow Step	Executor	Action/Description	Data (Input and/or Output)
6			Condition: Admin cancels deletion	
	1	Admin	Reviews confirmation dialog and clicks "Cancel" button instead of "Confirm Delete".	Input: Click action on Cancel button

	2	System	Closes the confirmation dialog without making any changes.	Input: None
	3	System	Returns admin to Player List page with all accounts unchanged.	Output: Player list table unchanged
	4	Use case ends (no deletion performed).		

UC-VAD1: View Analytics Dashboard

Use Case Name

View Analytics Dashboard

Use Case ID

UC-VAD1

Description

Goal

To allow a logged-in admin to view, analyze, and export comprehensive analytics and insights about player demographics, game performance, and system usage in the STARBOOKS system, enabling data-driven decision-making for system improvements and content planning.

Actions/Short Summary

- Admin navigates to Analytics Dashboard
- System retrieves and displays analytics data with visualizations
- Admin views various metrics and charts
- Admin applies filters to refine data views
- System updates visualizations based on filters
- Admin exports analytics data to Excel file for reporting

Trigger

Admin clicks on "Analytics Dashboard" menu item in the admin panel because they need to review system performance, analyze player trends, prepare reports, or make data-driven decisions about content and features.

Actor(s)

1. **Primary Actor** - Admin
2. **Secondary Actors** - System

Assumptions

1. The system is available and admin session is active.

2. Analytics data is aggregated and updated regularly.
3. System has sufficient data to generate meaningful insights.
4. Charts and graphs are interactive with drill-down capabilities.
5. Export functionality generates files.
6. Dashboard is read-only - no data modifications occur through analytics viewing.
7. System handles large datasets efficiently for quick dashboard loading.

Preconditions

1. Admin is logged into the admin panel (has active admin session).
2. Analytics data exists in the database.
3. System analytics aggregation service is operational.
4. Dashboard visualization library is loaded and functional.

Post-conditions

1. Admin has viewed analytics data and insights.
2. No changes are made to the database.
3. If exported, file is generated and downloaded.
4. System is ready to accept additional admin actions.

Scenario 1: Admin Successfully Views and Exports Analytics Data

Base Flow

Step	Executor	Action/Description	Data (Input or Output)
1	Admin	Navigates to admin panel and clicks on "Analytics Dashboard" menu item.	Input: Click action on Analytics Dashboard
2	System	Retrieves analytics data from database by running aggregation queries.	Input: Analytics data query; Output: Aggregated data (player counts, game statistics, demographic data, reward data)
3	System	Processes data to generate metrics and prepare visualizations.	Input: Raw analytics data; Output: Processed metrics and chart data
4	System	Displays Analytics Dashboard with multiple sections and visualizations.	Output: Dashboard display with sections: Summary Cards: Total Registered Players (count), Most Played Game Mode (name with percentage), Average Session Duration (time in minutes); Charts: Male vs. Female Registered Players (pie chart with percentages), Age Distribution of Players (bar chart with age ranges), Registered Players by Region (bar chart with region names), Male vs. Female

			Players Per Game Mode (grouped bar chart), Reward Distribution by Gender and Level (grouped bar chart by Easy/Average/Difficult), Most Played Game Mode by Age (grouped bar chart); Filter Panel: Date range picker, Region filter dropdown, Game mode filter dropdown, Age range filter, Gender filter, "Apply Filters" button, "Reset Filters" button; Export Button: "Export" icon/button
5	Admin	Reviews the displayed analytics data, scrolling through different sections and examining various charts.	Input: Visual review of data
6	Admin	Interacts with charts.	Input: Mouse interactions with charts; Output: Tooltips showing specific data points, drill-down details
7	Admin	Decides to apply filters to refine the view.	Input: Filter selections (date range, region, game mode, age range, gender)
8	Admin	Clicks "Apply Filters" button.	Input: Click action on Apply Filters
9	System	Validates filter selections and queries database with filter criteria.	Input: Filter parameters; Output: Filtered analytics data
10	System	Regenerates metrics and visualizations based on filtered data.	Input: Filtered data; Output: Updated metrics and charts
11	System	Updates dashboard display with filtered results, maintaining filter selections.	Output: Refreshed dashboard showing filtered data, applied filters displayed
12	Admin	Reviews filtered analytics data and identifies insights/trends.	Input: Visual analysis
13	Admin	Decides to export data for reporting and clicks "Export" icon/button.	Input: Click action on Export
14	System	Displays export options dialog.	Output: Export dialog (section checkboxes, export format, "Export" button, "Cancel" button)
15	Admin	Confirms export settings and clicks "Export" button.	Input: Click action on Export, export selections

16	System	Generates file containing analytics data with current filters applied.	Input: Analytics data, filter parameters; Output: Generated file with Summary, Player Demographics, Game Statistics, Reward Distribution
17	System	Initiates file download to admin's browser.	Output: File download prompt
18	System	Displays success message: "Analytics data exported successfully!"	Output: Success notification

Alternative Flows

Base Step	Alt Flow Step	Executor	Action/Description	Data (Input and/or Output)
2-4			Condition: No analytics data available	
	1	System	Queries database but finds no player data or game activity (new system, data cleared, etc.).	Input: Analytics query; Output: Empty result set
	2	System	Displays empty state message in dashboard: "No analytics data available yet. Data will appear once players register and start playing games."	Output: Empty state screen with helpful message and illustration
	3	System	May show sample/demo charts with dummy data labeled as "Sample Data - No Real Data Available".	Output: Sample dashboard (optional)
	4	Use case ends (no further interaction possible).		
9			Condition: Invalid filter selection returns no results	
	1	System	Applies filters but query returns no matching data (e.g., date range before system launch, region with no players).	Input: Filter parameters; Output: Empty result set
	2	System	Displays message in dashboard: "No results found for the selected criteria. Try adjusting your filters."	Output: Empty results message overlaid on dashboard

	3	System	Retains filter selections so admin can adjust them.	Output: Dashboard with filters visible, charts showing no data
	4	Admin	Adjusts filters or clicks "Reset Filters" to return to default view.	Input: Filter modification or reset
	5	Resume base flow at Step 8 or Step 4 (if reset).		
2-4			Condition: System error during data load	
	1	System	Attempts to retrieve analytics data but encounters database error, timeout, or service failure.	Input: Analytics query; Output: Error response
	2	System	Logs error for system administrators.	Output: Error log entry (timestamp, error details, query attempted)
	3	System	Displays error message: "Unable to load analytics data. Please try again. If the problem persists, contact support." with "Retry" button.	Output: Error message with retry option
	4	Admin	Clicks "Retry" button.	Input: Click action on Retry
	5	System	Reattempts to load analytics data (returns to Step 2 of base flow).	
16-17			Condition: Export fails	
	1	System	Attempts to generate file but encounters error (file generation library error, memory issue, data formatting error).	Input: Export request; Output: Export error
	2	System	Displays error message: "Export failed. Please try again. If the problem persists, try exporting a smaller date range."	Output: Error notification
	3	Admin	Can retry export with different settings or smaller data range.	Input: Retry with adjusted parameters
	4	Resume base flow at Step 13		

		(admin can retry export).		
--	--	---------------------------	--	--

UC-VLB1: Player View Leaderboard

Use Case Name

Player View Leaderboard

Use Case ID

UC-VLB1

Description

Goal

To allow a logged-in player to view the top-performing players in the STARBOOKS system.

Actions/Short Summary

- Player navigates to Leaderboard page
- System retrieves top players data from database
- System displays leaderboard
- Player reviews rankings and top performers
- Player views their own ranking

Trigger

Player clicks on "Leaderboard" menu item or navigation link in the game interface because they want to see top performers, check rankings, compare their performance with others, or find motivation to improve.

Actor(s)

1. **Primary Actor** - Player
2. **Secondary Actors** - System

Assumptions

1. The system is available and player session is active.
2. Leaderboard data is updated regularly.
3. Top 20 players are displayed.
4. Leaderboard rankings are based on total rewards claimed.
5. Leaderboard is read-only for players.
6. Ties are handled by earlier achievement date.

Preconditions

1. Player is logged into the system (has active session).
2. Player has valid account in the database.
3. Leaderboard data table exists and is accessible.
4. At least some game activity has occurred (or empty state is shown).

Post-conditions

1. Player has viewed leaderboard rankings.
2. No changes are made to the database (read-only operation).
3. System is ready to accept additional player actions.

Scenario 1: Player Successfully Views Leaderboards with Data

Base Flow

Step	Executor	Action/Description	Data (Input or Output)
1	Player	Clicks on "Leaderboard" link/button from the main navigation menu or homepage.	Input: Click action on Leaderboard
2	System	Retrieves leaderboard data from database for both game modes.	Input: Leaderboard query; Output: Top 20 players data (username, total rewards, rewards by difficulty Easy/Average/Difficult, last reward claim timestamp)
3	System	Checks if current player appears in either leaderboard.	Input: Current Player ID, leaderboard data; Output: Player's rank and position
4	System	Processes and sorts data for display.	Input: Raw leaderboard data; Output: Sorted, formatted leaderboard entries
5	System	Displays Leaderboard page	Output: Leaderboard page: Title "Top 20", Table columns (Rank #, Username, Total Rewards, Easy Rewards, Average Rewards, Difficult Rewards, Last Claim Date), Rows 1-20 with player data, Current player's row highlighted if in Top 20; Player's Own Stats (if not in Top 20): "Your Rank: #[X]" displayed below each leaderboard section
6	Player	Reviews leaderboard standings, comparing top performers and checking their own position.	Input: Visual review of rankings
7	Player	May scroll through both leaderboards to see all Top 20 entries.	Input: Scroll action

8	Player	Identifies top players and performance benchmarks for motivation.	Input: Mental analysis of data
9	Player	Can navigate away from leaderboard when finished reviewing.	Input: Click action on navigation link or back button

Alternative Flows

Base Step	Alt Flow Step	Executor	Action/Description	Data (Input and/or Output)
2-5			Condition: No leaderboard data available	
	1	System	Queries database but finds no game activity.	Input: Leaderboard query; Output: Empty result set
	2	System	Displays empty state message: "No leaderboard data available yet. Be the first to play and earn your spot on the leaderboard!"	Output: Empty state screen with encouraging message and illustration
	3	Use case ends (no leaderboard data to view).		
2-4			Condition: System error while loading leaderboard	
	1	System	Attempts to retrieve leaderboard data but encounters database error, timeout, or service failure.	Input: Leaderboard query; Output: Error response
	3	System	Displays error message: "Unable to load leaderboard. Please try again later." with "Retry" button.	Output: Error message with retry option
	3	Player	Clicks "Retry" button.	Input: Click action on Retry
	4	System	Reattempts to load leaderboard data (returns to Step 2 of base flow).	

UC-VLB2: Admin View Leaderboard

Use Case Name

Admin View Leaderboard

Use Case ID

UC-VLB2

Description

Goal

To allow a logged-in admin to view, analyze, and export leaderboard data for Whiz Challenge and Whiz Battle game modes in the STARBOOKS system, enabling performance monitoring, competitive analysis, and reporting capabilities.

Actions/Short Summary

- Admin navigates to Leaderboard page in admin panel
- System retrieves and displays leaderboard data
- Admin reviews top performers and rankings
- Admin optionally applies filters or sorting
- Admin exports leaderboard data for reporting
- System generates and downloads file

Trigger

Admin clicks on "Leaderboard" menu item in the admin panel because they need to review competitive rankings, identify top performers, prepare reports, monitor game engagement, or export leaderboard data for analysis.

Actor(s)

1. **Primary Actor** - Admin
2. **Secondary Actors** - System

Assumptions

1. The admin has internet connectivity.
2. Leaderboard data is updated regularly.
3. Admin view shows Top 20 by default.
4. Admin has export functionality not available to players.
5. Export feature includes full leaderboard data with timestamps.
6. Admin can view complete data including player IDs and detailed timestamps.
7. Leaderboard view is read-only (no modifications to rankings).

Preconditions

1. Admin is logged into the admin panel (has active admin session).
2. Leaderboard data table exists and is accessible.
3. At least some game activity has occurred (or empty state is shown).

Post-conditions

1. Admin has viewed leaderboard rankings.
2. No changes are made to the database (read-only operation).
3. If exported, file is generated and downloaded.
4. System is ready to accept additional admin actions.

Scenario 1: Admin Successfully Views and Exports Leaderboard Data**Base Flow**

Step	Executor	Action/Description	Data (Input or Output)
1	Admin	Navigates to admin panel and clicks on "Leaderboard" menu item.	Input: Click action on Leaderboard
2	System	Retrieves leaderboard data from database for both game modes.	Input: Leaderboard query; Output: Top 20 players data (Player ID, username, total rewards, rewards by difficulty Easy/Average/Difficult, last reward claim timestamp, reward claim count)
3	System	Processes and sorts data for display (rankings 1-20).	Input: Raw leaderboard data; Output: Sorted, formatted leaderboard entries with admin-specific details
4	System	Displays Leaderboard page in admin panel with two sections and export controls.	Output: Admin leaderboard page with sections: Title "Top 20", Table columns (Rank #, Player ID, Username, Total Rewards, Easy Rewards, Average Rewards, Difficult Rewards, Last Claim Date, Total Claims), Rows 1-20 with player data; Controls: "Export" button
5	Admin	Reviews leaderboard standings, examining top performers and their statistics.	Input: Visual review of rankings
6	Admin	May apply filters or adjust view settings (e.g., change date range, filter by region).	Input: Filter selections (optional)
7	System	If filters applied: updates leaderboard display based on filter criteria.	Input: Filter parameters; Output: Filtered leaderboard data

8	Admin	Decides to export data for reporting and clicks "Export" button (for specific leaderboard or "Export All").	Input: Click action on Export
9	System	Displays export options dialog.	Output: Export dialog
10	Admin	Selects export options and clicks "Export" button.	Input: Click action on Export, export selections
11	System	Retrieves complete leaderboard data based on export selections.	Input: Export parameters; Output: Complete leaderboard dataset
12	System	Generates file with formatted sheets.	Input: Leaderboard data; Output: Generated file
13	System	Initiates file download to admin's browser.	Output: File download prompt
14	System	Displays success message: "Leaderboard data exported successfully!"	Output: Success notification

Alternative Flows

Base Step	Alt Flow Step	Executor	Action/Description	Data (Input and/or Output)
2-4			Condition: No leaderboard data available	
	1	System	Queries database but finds no game activity for either Whiz Challenge or Whiz Battle.	Input: Leaderboard query; Output: Empty result set
	2	System	Displays empty state message: "No leaderboard data available yet. Data will appear once players start playing games."	Output: Empty state screen with informational message
	3	System	Disables export functionality (no data to export).	Output: Export button disabled with tooltip "No data available"
	4	Use case ends (no leaderboard data to view or export).		

2-4			Condition: System error while loading leaderboard	
	1	System	Attempts to retrieve leaderboard data but encounters database error, timeout, or service failure.	Input: Leaderboard query; Output: Error response
	2	System	Logs error for system administrators.	Output: Error log entry (timestamp, error details, query attempted)
	3	System	Displays error message: "Unable to load leaderboard data. Please try again." with "Retry" button.	Output: Error message with retry option
	4	Admin	Clicks "Retry" button.	Input: Click action on Retry
	5	System	Reattempts to load leaderboard data (returns to Step 2 of base flow).	
11-13			Condition: Export fails	
	1	System	Attempts to generate file but encounters error	Input: Export request; Output: Export error
	2	System	Displays error message: "Export failed. Please try again. If the problem persists, try exporting a smaller dataset."	Output: Error notification
	3	Admin	Can retry export with different settings.	Input: Retry with adjusted parameters
	4	Resume base flow at Step 8 (admin can retry export).		

